

1.00 Introduction

The P?S Enhanced PDP-8 Simulator (PEPS) is a software-based simulation of a 12-bit Digital Equipment Corporation (DEC) PDP-8 series computer. The package includes support for many of the peripheral devices found within real-world hardware configurations; certain PEPS package components were created by various members of the P?S^[1] on the original hardware.

The overall simulated environment is determined by specific initialization and configuration files controlling the basic "engine" of the PEPS package: the SIMH^[2] simulator for the DEC PDP-8/E and similar computers.

This document serves as the primary user guide to all features of the PEPS package. Unless otherwise indicated, default PEPS configuration files are assumed to be in effect; this enables all PDP-8 devices^[3] required for either entry-level (paper-tape-oriented) programs or any of several provided single-user storage-based operating systems as described below in section 1.10.

While the SIMH simulator can be used to create a minimal paper-tape environment, emphasis is given to larger systems that can support any of several included single-user storage-based operating systems. This allows execution of more complex programs, some of which are specific to the operating system environment chosen.

If desired, new programs can be created using a variety of development techniques. In the PEPS environment, program development focuses on using facilities available on the host Microsoft Windows platform. Depending on the user's needs, this can include creation of output files meant to be viewed in the host environment; in specific cases, various forms of post-processing can be applied for the purpose of creating output suitable for printing on windows-compliant printers or viewing on-screen with commonly available document-processing programs (such as Adobe Reader). PEPS package usage has included development of PDP-8 assembly language listings of well-organized source code; in specific instances this has produced fully-formatted listing files of programs previously unavailable (other than in the primitive form of badly copied paper listings that limits any meaningful usage).

This user guide, together with the documents referenced in various appendices, provides information on the installation, maintenance and operation of the PEPS package.

System requirements and initial installation.

The PEPS package is designed to work on any release^[4] of Microsoft Windows XP, Windows Vista, Windows 7, Windows 8 or Windows 10. It is generally distributed as a self-extracting archive capable of being installed on any permanently lettered logical drive. Due to internal restrictions, the PEPS package cannot be located on network shares or removable drives where the logical drive letter is subject to change.

Appendix A describes the installation procedure, including the ability to create custom descendent installation packages should the user wish to include additional files or PEPS features beyond the standard release. All details of the installation process must be closely followed in order to achieve a usable result.

Logical drive considerations.

While not widely known, the use of multiple logical drives in Windows is actually encouraged by Microsoft. This is best accomplished using various third-party hard drive partitioning programs; however, certain Windows releases include Administrative Tools capable of performing a rudimentary form of partitioning that may be adequate for some users.

Appendix C describes several third-party partitioning programs that are compatible with all operating systems that support the PEPS package. These utilities also include additional features that may be of use to advanced users beyond the scope of logical drive placement of the PEPS package.

Note: For illustrative purposes only, the PEPS package elements will be described throughout this document as if installed on Windows logical drive P:. Any permanently lettered logical device may be used; however, it is recommended the PEPS package be installed on a dedicated drive to facilitate backup of working copies of the package.

Additional required utilities not supplied with the PEPS package.

Certain auxiliary programs compatible with Windows are likely required depending on the user's needs; Appendix C contains descriptions of many of these additional applications. Among the utilities presented are various text editors known to be compatible with the PEPS package and post-processing programs to facilitate viewing or printing text files created by PEPS (such as assembly listings of programs under development).

Most of the PEPS documentation files (including this user guide) are distributed in Portable Document Format (PDF). As such, a compatible document-processing program (such as Adobe Reader) must be installed prior to the full usage of the PEPS package.

The rest of this document assumes the PEPS package and all necessary third-party applications are already installed and ready to use; as the user gains familiarity with the package, minor adjustments may be implemented as necessary. In particular, calls to the installed PDF document reader must be updated in certain BATCH script files to reflect the current installation; calls to other utilities must be updated in an analogous manner where relevant. Appendix C contains more information on these utilities and the interaction with the associated script files. Examples will be provided for calling common programs from within both 32-bit and 64-bit operating systems.

Note: It is recommended that all relevant considerations described above be reconciled and fully understood before proceeding.

1.10 General Description

The PEPS package provides a functional simulation of a PDP-8 system with an array of available peripherals. When compared to real-world hardware, the included configurations are considered to span the range from entry-level through large-scale systems. The package provides much of the performance of original PDP-8 systems while actually running on a windows-based personal computer through software simulation.

As typically configured, the following PDP-8 resources are generally available within the PEPS environment:

a) PDP-8/E Central Processing Unit (CPU)

Note: The KK8F CPU is used in several DEC models such as the PDP-8/E, PDP-8/F, PDP-8/M and PDP-8/A-620 series models; no software method can determine which of these models is in use. Common practice is to refer to the PDP-8/E since this was the first model produced including this hardware.

b) KE8E extended arithmetic element (EAE)

Note: with the EAE option installed, an improvement in model identification occurs. If a machine is determined to be a PDP-8/A, it could be referred to as the PDP-8/A-625; however, as described above, common practice still refers to the PDP-8/E (with EAE).

c) 32K of main memory

The SIMH simulator supports commands to deliberately limit memory size to smaller amounts by 4K field increments (analogous to the manner in which real-world machines can be configured). (Note: Certain supported operating systems include commands to deliberately ignore available memory. This is occasionally used to protect the contents of the upper portions of memory in specific circumstances.

Many system-specific programs need to be tested on smaller configurations to confirm proper operation of dynamic allocation routines for all supported instances of memory size. For example, the various PAL language assemblers require additional memory to assemble larger programs as a function of symbol count. Developing such programs requires that all supported memory sizes function as intended. Each supported operating system accomplishes memory size adjustment by system-specific commands; consult the appropriate software manuals for further details.

There is some overlapping of memory size limitations between the SIMH simulator initialization files and the internal features of the various operating systems; however, changing the initialization files for SIMH then requires initiating a new SIMH session. The changes brought about by the system-specific commands can generally be accomplished either immediately or by at most a restart of the operating system. There are situations where it is useful to boot to more than one operating system within a single SIMH session, perhaps to perform inter-system file conversions.

Note: By adjusting certain parameters within the configuration files, the availability of EAE, memory size and other particulars may be changed. Appendix Q contains more information on related SIMH parameters which may be of interest to advanced users.

d) System console terminal display device (device 03/04)

While original real-world PDP-8 systems generally used the Teletype Model 33 Automatic Send and Receive (ASR) terminal as the system console, this implementation is fully compliant with more recent devices (such as the VT05 display terminal) with regard to handling `<HT>` characters visually aligned to multiples of eight spaces. Depending on operating system specifics, console terminal output may use `<HT>` as described here, or simulate the display using a calculated number of `<SP>` characters.

In many aspects the console display resembles the output of a Teletype Model 33 printing unit using pale yellow roll paper and bold black ink; this represents a compromise between the original teletype output appearance and display devices found on newer systems well after paper-tape software became obsolescent.

Note: For operating systems such as OS/8^[5] with known defects in console terminal handling, the console simulation will be restricted to forced upper-case input and output, as well as suppression of superfluous output of certain control characters which are known to have visual representation in the host windows environment.

e) System lineprinter (device 66)

Standard PDP-8 lineprinters (such as the LP08) print on fan-fold paper up to ANSI B (Ledger/Tabloid) size of 11" x 17". This allows 66 lines and up to 132 columns per page. This device is generally referred to as LPT:. Using post-processing techniques available within the PEPS package, LPT: output can be created in PDF format.

Note: For certain configurations, the lineprinter device will be deliberately disabled. If this is the case, attempting to output to device 66 will not raise the done flag leading to device time-out.

f) PC04 high-speed reader/punch (device 01/02)

The high-speed reader is generally referred to as the PTR: device. All high-speed reader input is setup in the host environment for the next SIMH session; after all character frames have been read, further attempts to obtain input from the PTR: device will result in device time-out.

The high-speed punch is generally referred to as the PTP: device. High-speed punch output is cumulative across SIMH sessions.

Note: Individual file transfer from the PEPS host environment to the SIMH environment is achieved by using the PTR: device. File transfer from the SIMH environment to the PEPS host environment is achieved by using either the PTP: device or the LPT: device.

g) TC08 DECTape controller (device 76-77)

The TC08 DECTape controller can be configured with up to eight DECTape drive units numbered 0 through 7. Each logical drive is configured by association with an active compatible device file which represents the contents of a specified DECTape drive. If the associated drive file does not exist, it will be created automatically when attempting to access the logical drive unit.

The TC08 DECTape controller is based on the TC01 DECTape controller. While there are real-world differences between the two devices, all standard TC08 DECTape software runs on either device.

Note: Attempting access to a non-configured logical drive will cause a SELECT error within the TC08 controller; each supported operating system is responsible for handling all errors. Certain interrupt-driven application programs will not function properly if this condition occurs and the associated error flag is not cleared. Should this situation become a problem, configuring all logical units to be associated with active drive files will remedy the situation. (Default configuration files supplied with the PEPS release properly configure all logical drives for the TC08 and other devices. Advanced users can easily modify the configuration files to add support for custom device files as required)

h) Primary TD8E controller (device 77)

The TD8E DECTape controller can be configured with up to two DECTape drive units numbered 0 through 1 with the same file considerations for the TC08 controller as described above in g).

The primary TD8E controller conflicts with the TC08 controller due to overlapping device codes; as such they are mutually exclusive. Three additional TD8E Controllers are defined as follows:

- i. The secondary TD8E controller (device 76) can be configured with up to two DECTape drive units numbered 2 through 3 with the same file considerations for the TC08 controller as described above in g).

The secondary TD8E controller conflicts with the TC08 controller due to overlapping device codes; as such, they are mutually exclusive.

- ii. The tertiary TD8E controller (device 75) can be configured with up to two DECTape drive units numbered 4 through 5 with the same file considerations for the TC08 controller as described above in g).

The tertiary TD8E controller conflicts with the primary RX8E controller described below due to overlapping device codes; as such, they are mutually exclusive.

- iii.* The quaternary TD8E controller (device 74) can be configured with up to two DECTape drive units numbered 6 through 7 with the same file considerations for the TC08 controller as described above in g).

The quaternary TD8E controller conflicts with the RK8E controller described below due to overlapping device codes; as such, they are mutually exclusive.

Note: At the present time, the SIMH simulator only supports one of the TD8E controllers in any given configuration. As such, certain specific TD8E configurations available on real-world systems cannot be realized on the SIMH simulator.

By design, the SIMH simulator cannot simultaneously support the TC08 controller and any TD8E controller despite the fact the tertiary and quaternary TD8E controllers are not in device code conflict with the TC08 controller. Pertinent real-world systems are generally configured in an analogous matter; as such, this is not a serious consideration. Additionally, such configurations are in conflict with either the RX8E controller or the RK8E controller as described below.

i) RK8E removable cartridge disk controller (device 74)

The RK8E removable cartridge disk controller can be configured with up to four RK05 drive units numbered 0 through 3. As in the case of DECTape drives, should the associated drive file not exist, it will be created automatically when first accessed.

Note: A modified controller known as the RKS8E allows up to eight RK05 drives. However, this device is not supported by SIMH at this time. Each supported operating system must be configured to avoid anomaly with regard to misinterpreting the actual configuration.

j) Primary RX8E controller (device 75)

The RX8E controller can be configured with up to two RX01 drive units numbered 0 through 1. As in the case of RK05 or DECTape drives, should the associated drive file not exist, it will be created automatically when first accessed.

A secondary RX8E controller (device 76) is defined; however, device conflicts exist with both the TC08 DECTape controller and the secondary TD8E controller. As such, most configurations do not support this device.

Note: Various real-world systems (such as the VT78) support an extended primary RX8E-compatible controller that can be configured with an additional RX01/RX02 drive pair. The SIMH simulator does not currently support these device extensions.

Additional devices may be available depending on enabling specific configuration parameters of the SIMH simulator. Appendix D includes a list of these devices; however, specific software drivers and/or additional utility programs may be required for proper support.

Note: Limitations of the SIMH environment prevent dismountable devices (such as DECTapes, RK05 and RX01 drives) from being loaded or unloaded during SIMH sessions

One of the design goals of the PEPS package is to allow advanced users the ability to modify the current configuration as easily as possible. However, the SIMH simulator must be restarted after any changes are made to the configuration files.

Wherever possible, the various components of the SIMH simulator are maintained in separate directories to avoid clutter and needless confusion. Emphasis is given to accessing configuration files that determine the exact simulation during any SIMH session.

Any number of configurations can be made available in a PEPS installation; however, only one configuration can be run at any given time. This allows maximum flexibility in terms of mounted device specifics and operating system configurations easily accessible from a single directory.

Future releases of the PEPS package may support additional operating systems; the initial release only supports OS/8 and P?S/8 in various configurations; these are the two systems deemed most useful for most PDP-8 usage. In specific instances, it may prove to be difficult to support other systems without the relevant source code files; the files may be difficult to obtain and/or were intended to be developed on other hardware systems such as the PDP-10 TOPS10 operating system. Those desiring to support these systems are encouraged to gain proficiency with both OS/8 and P?S/8 program development features, since a combination of utilities available in both systems may be required to allow proper support entirely based within the PEPS environment. In general, the P?S/8 PAL assembler is a superset of OS/8 PAL8 and supports nearly all TOPS10 PAL10 features that might apply.

1.20 General Usage

Note: PDP-8 software supports several different file structures, all referred to in the SIMH simulator environment as directories. In windows, the analogous structures are often referred to as folders. Within this user guide, the terms will be used interchangeably only when referring to the file structures of Microsoft windows.

Using the PEPS package typically starts by opening the PEPS Command directory on the windows desktop; this serves as the "command center" for most PEPS operations. From this directory, nearly all PEPS functions and utilities can be initiated. This includes bringing up the SIMH PDP-8 simulator booting to one of several supported operating system configurations and various file-related functions such as file editing and file maintenance in the host windows environment. The Command directory is detailed in section 1.50 below.

The user must have a general understanding of Microsoft windows to make use of many of the available features of the PEPS package. Knowledge of the PDP-8 operating systems is essential to use the various facilities provided and, if desired, help to draw comparisons between the supported operating systems.

Appendix B includes references to documents for the various supported PDP-8 operating systems which can aid in their usage, many of which are provided only in the full PEPS distribution which will not be widely distributed in part due to sheer size.

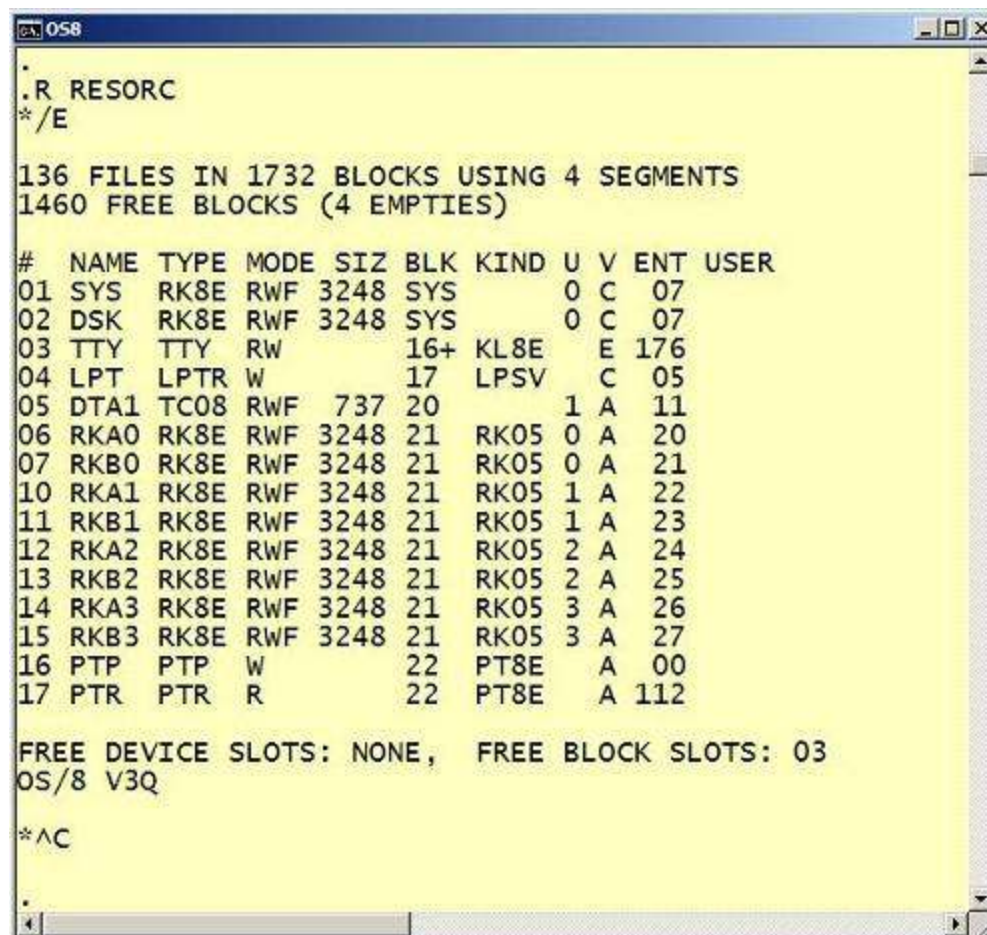
More comprehensive releases are intended for developers of either the PEPS package itself or any of the supported operating systems, especially P?S/8. Any questions regarding specific extensions to the PEPS package beyond the public release are encouraged to contact the author, Charles Lasner, who is also the principal architect of P?S/8.

PUT PAPER-TAPE STUFF HERE!!!!!!!

1.30 Supported PDP-8 Operating Systems

1) OS/8 Version V3D (V3Q)

Figure 1 shows console output specific to the OS/8 *RESORC* utility used to display the current configuration of supported devices.

A screenshot of a terminal window titled "OS8". The window displays the output of the ".R RESORC" command. The output shows system statistics and a table of resources. The statistics include 136 files in 1732 blocks using 4 segments, and 1460 free blocks (4 empties). The table lists 17 resources with columns for ID, NAME, TYPE, MODE, SIZE, BLK, KIND, U, V, ENT, and USER. At the bottom, it shows "FREE DEVICE SLOTS: NONE, FREE BLOCK SLOTS: 03" and "OS/8 V3Q". The prompt "*^C" is visible at the bottom.

```
.R RESORC
*/E

136 FILES IN 1732 BLOCKS USING 4 SEGMENTS
1460 FREE BLOCKS (4 EMPTIES)

#  NAME TYPE  MODE  SIZ  BLK  KIND  U  V  ENT  USER
01 SYS  RK8E  RWF   3248  SYS      0  C   07
02 DSK  RK8E  RWF   3248  SYS      0  C   07
03 TTY   TTY   RW      16+  KL8E     E  176
04 LPT  LPTR   W      17   LPSV     C   05
05 DTA1 TC08  RWF    737  20      1  A   11
06 RKA0 RK8E  RWF   3248  21     RK05  0  A   20
07 RKB0 RK8E  RWF   3248  21     RK05  0  A   21
10 RKA1 RK8E  RWF   3248  21     RK05  1  A   22
11 RKB1 RK8E  RWF   3248  21     RK05  1  A   23
12 RKA2 RK8E  RWF   3248  21     RK05  2  A   24
13 RKB2 RK8E  RWF   3248  21     RK05  2  A   25
14 RKA3 RK8E  RWF   3248  21     RK05  3  A   26
15 RKB3 RK8E  RWF   3248  21     RK05  3  A   27
16 PTP  PTP   W      22   PT8E     A   00
17 PTR  PTR   R      22   PT8E     A  112

FREE DEVICE SLOTS: NONE,  FREE BLOCK SLOTS: 03
OS/8 V3Q

*^C
```

Figure 1. OS/8 Supported Resource

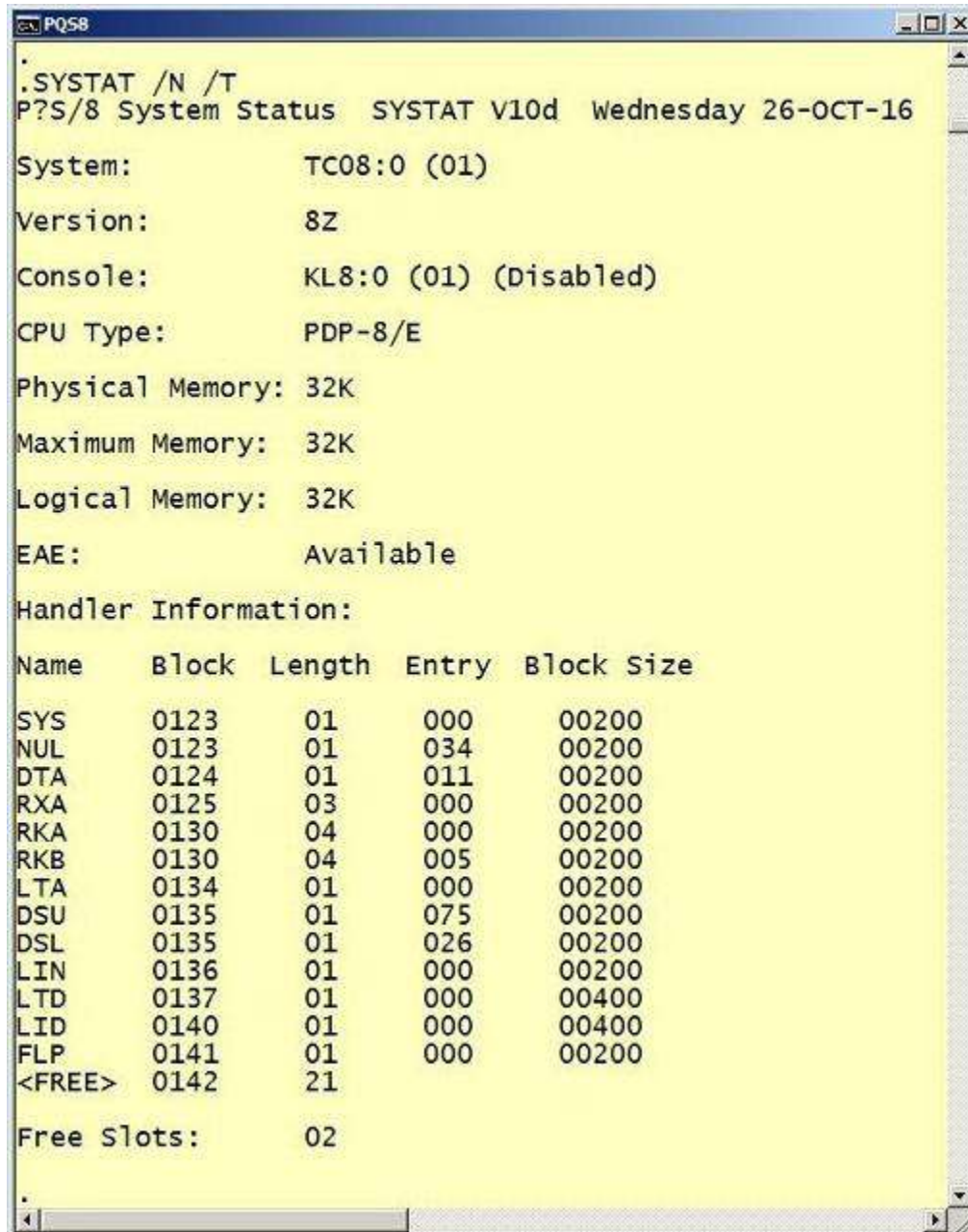
Note: The exact configuration for any OS/8 system can be modified using the OS/8 *BUILD* program as long as all devices are supported by the SIMH simulator. Several renamed alternate configuration copies of BUILD are provided in the PEPS implementation of OS/8 to

allow creation of the most common variations.

2) P?S/8^[6] Version 8Z

Shown below in Figure 2 is console output specific to the P?S/8 SYSTAT utility used to display various configuration details and specifics of the hardware including system memory size and whether EAE is installed.

All currently installed non-system device handlers are detailed.
(Note: As of P?S/8 Version 8Z, the only system program capable of loading non-system device handlers is P?S/8 BLKCPY; this utility can copy [with optional verification] or compare groups of adjacent blocks between any two supported devices.)



```
.SYSTAT /N /T
P?S/8 System Status  SYSTAT V10d  Wednesday 26-OCT-16

System:          TC08:0 (01)
Version:         8Z
Console:         KL8:0 (01) (Disabled)
CPU Type:        PDP-8/E
Physical Memory: 32K
Maximum Memory:  32K
Logical Memory:  32K
EAE:             Available

Handler Information:

Name   Block   Length  Entry  Block Size
-----
SYS    0123     01     000    00200
NUL    0123     01     034    00200
DTA    0124     01     011    00200
RXA    0125     03     000    00200
RKA    0130     04     000    00200
RKB    0130     04     005    00200
LTA    0134     01     000    00200
DSU    0135     01     075    00200
DSL    0135     01     026    00200
LIN    0136     01     000    00200
LTD    0137     01     000    00400
LID    0140     01     000    00400
FLP    0141     01     000    00200
<FREE> 0142     21     000    00200

Free Slots:      02
```

Figure 2. P?S/8 System Status and Available Resources

Further information on using the supported operating systems is provided in section 1.61 below.

Note: While it is possible for certain PDP-8 programs to run from nearly all available operating systems merely by strict adherence to the prevailing memory conventions, each operating system supports specific unique features that require complete program conformance for proper operation. This will be discussed in greater detail in section 1.61 below. Appendix R contains documentation of all important considerations with regard to program conformance with the various operating systems.

1.40 Using the SIMH simulator for program development

While it is possible to execute existing PDP-8 programs within the SIMH environment, the full implementation (including the PEPS extensions) allows for meaningful development of any PDP-8 program; in many cases, work flow in the PEPS environment exceeds that of the original hardware configuration. Various operating modes are described below.

1.41 Using the SIMH simulator for basic PDP-8 programming

The SIMH PDP-8 simulator has been developed over the course of many years, gradually adding in numerous features and functionality beyond the original implementation. Earlier releases generally concentrated on mimicking the most primitive PDP-8 systems, which only had a limited capability generally referred to (somewhat euphemistically) as the *paper-tape operating system*.

In actuality, this is merely a series of inter-related binary files stored on BIN^[7] format binary paper-tapes intended to be loaded into memory from either the high-speed reader or the low-speed reader attached to either the Teletype Model 33 ASR or Model 35 ASR terminal.

Software commands were then added to the SIMH simulator as a stand-in for a hardware front panel; at that point, the scope of what could be accomplished was limited to what could be done with the *paper-tape operating system* components and any additional binary paper-tapes created by the user. This situation can best be described as a tedious waste of development time due to the frustrating changes of contents of memory carrying out each of the many steps; each step would take significant time and effort. The programmer's thought process regarding the details of the actual program being developed was often interrupted by the overhead of moving onto the next small step in a very long chain of events that accomplished very little.

The issues associated with the *paper-tape operating system* are actually intensified when using the SIMH simulator as there is maximum dependency on the internal commands added in lieu of a hardware front panel. Each component program requires the use of the paper-tape BIN loader to have it loaded into memory, followed by inspection of the accumulator register (AC) when the loader halts to ensure proper loading of the latest utility. Assuming no errors in loading, the latest component must be started at a designated address with a different manual operation.

Generally, the front panel switches must be set to specific values to inform the program of the general nature of the current usage of the program; this in turn could lead to several passes in the case of an assembler program (generally the primitive PAL III assembler) with the CPU halting after each pass over an input source file tape.

Paper-tape editing programs must also be loaded and started; during the editing session, the paper-tape source file must be read in and also a descendant copy punched. Assuming a nominal successful assembly in several passes leading to a punched binary paper-tape of the program being developed, this program binary paper-tape must also be loaded by the BIN loader, etc. Each context change of memory requires several commands entered into the running SIMH session to either monitor or carry out relatively small operations at the expense of maximum programmer/operator effort and potential for tedium.

While this primitive capability still remains within the SIMH simulator, the addition of a large array of storage and other peripherals allows meaningful usage of more complex systems. Software operating systems that are booted up from the supported devices allows any level of user to accomplish much more than just a basic system (whether real-world based or simulated). In typical real-world systems, program development that using a storage-based operating system could be accomplished in perhaps a matter of hours could take months to accomplish when limited to the paper-tape operating system components.

For those who only want to use this limited capability, two alternatives are possible: a) The tedious process as described above involving constantly changing the SIMH configuration at each step, b) Using a few commands from the supported operating systems to eliminate several of the steps requiring more intense management. Minimal knowledge of the supported operating systems allows use of the basic software without experiencing some portion of the associated tedium. If the simulated environment merely consists of a CPU with memory and a console terminal (generally a Teletype Model 33 ASR) and perhaps a PC04 high-speed reader and punch, this predictably leads to user frustration as is also the case on a real-world equivalent hardware system.

Both the P?S/8 and OS/8 operating systems contain commands to support the rudiments of what can aid in supporting this primitive software environment. Appendix F details the specific programs used to support paper-tape operations available in both storage-based operating systems; only a small number of easily-learned commands are required to fully support improving the limited capability of the paper-tape environment.

While the specific programs used in the paper-tape environment are not provided with the PEPS package, they are available from various Internet-based PDP-8 archive libraries. Due to the nature of the overall PEPS environment, these programs may be added to allow easy deployment as required; some additional knowledge of the storage-based operating systems may be required. In general, the storage-based operating systems provide superior equivalents to these programs; as such, these earlier programs are not included in the PEPS package.

1.42 Using the full PEPS environment for PDP-8 programming

As compared to the paper-tape environment, either of the two provided operating systems (OS/8 and P?S/8) can be called up to perform far more sophisticated operations including running additional programming languages, file editors and other utilities unique to each operating system. Each system is capable of self-contained independent operation including program development. Commands within the PEPS Command directory are available to initiate these operations, as well as transmit files from the host Windows environment to the SIMH environment or vice-versa.

The two operating systems can also exchange the contents of files using available conversion programs, most notably the P?S/8 *OS8CON* utility used for interchanging text files.

Binary files can be transferred between the two operating systems using various methods including binary paper-tape punching in one system followed by binary file reading in the other. The Command directory includes a shortcut/icon to an internal routine to swap the *PTRIN* and *PTPOUT* files for this specific purpose. Note: Future releases of the P?S/8 operating system will include an additional utility to be known as *BINCON* patterned after *OS8CON* to more directly effect binary data transfers between the two operating systems. This will allow OS/8 users the ability to more readily utilize the binary output of the superior P?S/8 *PAL* assembler program. *PAL* includes features not available within OS/8's *PAL8* utility including additional language features, advanced support of the LINC-8 and PDP-12 LINC processor and corrected support of the DTORG directive to allow meaningful usage of this feature in both operating systems.

Configurations of the two operating systems are each capable of booting up the other from within the same SIMH session. Appendix G documents the available methods to transfer files between operating systems (some of which are currently under development as of this writing). By using the best features of both systems, the PEPS package can accommodate any level of usage and development realizable in a PDP-8-based environment.

Appendix F illustrates a minimal paper-tape session for those who want to experience this minimal usage of the PEPS package. Some of the paper-tape software programs are being added to the PEPS package in a dedicated folder for the purpose. The system manifest file should be checked for a current list of included files.

1.43 Host editing and development of PDP-8 source files

While the prevailing operating systems are capable of independent source file development, a distinctive feature of the PEPS environment is the support of host-level file editing with rapid transmission of updated files into the SIMH environment; multiple rounds of editing sessions and program assembly or execution (in the case of high-level languages such as FOCAL) can quickly be achieved. While the standard Windows Notepad editor is the default editing program for the supplied editing commands, any completely conforming alternative editing program can be used. Appendix D contains information on some of these editing programs and certain tradeoffs involved when using them.

There are two basic approaches to performing host editing and development of source files:

1) Easily specify a unique filename limited to a generic directory using pre-existing generic commands; only one designated file at a time can be used with this method.

The generic *Current_Program_Files* directory can contain any number of unique filenames; selecting one for development allows access to a set of available commands already included in the package.

This process is documented in section 1.50 below regarding commands related to the *Current_Program_Files* directory such as Set Current Program and Edit Current Program.

An additional feature of the PEPS package is to readily work with any one file in the generic *Current_Program_Files* directory. The only limitation is that access is limited to the currently designated file as the commands are generic to all such files. This process is documented in section 1.50 below regarding commands related to the *Current_Program_Files* directory such as Set Current Program and Edit Current Program.

2) Use a pair of custom commands specific to a designated project file; any number of unique command pairs for specific files can be defined.

As described in 2) above, for any programming project based on a particular file, two icon-based command shortcuts are required specific to that file. Any number of such projects can be defined and be in different stages of development at the same time.

It is recommended only advanced users attempt to create their own command pairs. The process requires significant knowledge of arcane features of Microsoft Windows including working with file attributes from the windows environment (and also from a command prompt) and managing environment variables as used in the internal properties of windows links/shortcuts. Appendix H details the process using one of the provided file projects as an example.

An additional feature of the PEPS package is to readily work with any one file in the generic *Current_Program_Files* directory. The only limitation is that access is limited to the currently designated file as the commands are generic to all such files. This process is documented in section 1.50 below regarding commands related to the *Current_Program_Files* directory such as Set Current Program and Edit Current Program.

Any user of the PEPS package can easily work with any number of files, changing focus between active files as necessary. If the generic commands method is used, the only restriction is the use of a single file at a time and limiting file storage to a specific directory. This process is documented in section 1.50 below regarding commands related to the *Current_Program_Files* directory such as Set Current Program and Edit Current Program.

The advantage for this section is setting up a particular file for development can be accomplished quickly and easily; the particular file method requires considerable effort to create the command shortcuts.

Any user of the PEPS package can easily work with any number of files, changing focus between active files as necessary. If the generic commands method is used, the only restriction is the use of a single file at a time and limiting file storage to a specific directory. This process is documented in section 1.50 below regarding commands related to the *Current_Program_Files* directory such as Set Current Program and Edit Current Program.

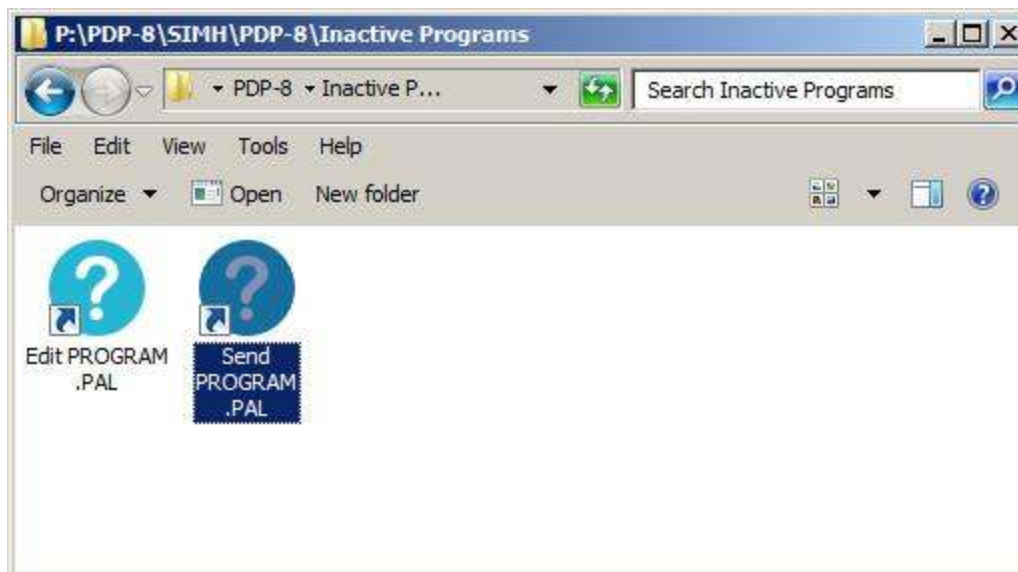


Figure 3. Inactive Programs directory

As shown in Figure 3 above, the *Inactive Programs* directory (accessible from the main Command directory) contains a pair of command shortcut/links that can be moved to the main Command directory as necessary; this can be accomplished using standard windows Explorer techniques such as Cut or Copy and Paste. These example commands are completely operational; the associated command files (Edit and Copy & the SIMH Batch Files directory as the PEPS package requires.

Note: Due to specific quirks of windows, embedded space characters may be placed within the shortcut text to achieve better visual results. This is often a trial and error process specific to the display method chosen; in this example medium-sized icons are used. Other display options such as *List* or *Details* or *Tiles* will tend to have different requirements.

This example is particular to the project file named *PROGRAM.PAL*. Other projects can be defined using alternate file names. There is no practical limit to the number of independent project files; however, the creation of shortcut/links and underlying support files is an advanced process to be accomplished by advanced users of the PEPS package. Appendix H details the underlying batch files that implement the *PROGRAM.PAL* file project including the arcane details of windows that must be followed for a successful project.

Note: Particular releases of the PEPS package may include additional command/shortcut links and associated support files; these files are provided by other PEPS package users to assist in creating new custom project files.

The use of the *PROGRAM.PAL* project is described as follows:

- 1) Edit *PROGRAM.PAL*. This command brings up the source file `\PDP-8\Current_Program_Files\PROGRAM.PAL` for editing in the windows *Notepad* program. Editing changes can be made at any time. Either saving the file within the editor or exiting the editor is necessary to proceed in the development process of this file.

Note: Appendix C contains information regarding conforming alternate editing programs which can be substituted for windows Notepad on an individualized basis, as well as potential feature tradeoffs that may result. These considerations apply to all file editing situations throughout the PEPS environment. It is strongly recommended only advanced users modify the editing program parameters to call up any particular file editing alternative due to potential interaction with open files and other running tasks (such as the SIMH simulator). In certain instances, the host Windows release (32-bit or 64-bit) may also be a factor.

If the editor is closed, the contents of the PROGRAM.PAL file is copied to the *PTRIN.TXT* file which, in turn, is used as the dedicated input to the simulated paper-tape reader device (PTR:) within the SIMH simulator environment. Once this is accomplished, any of the supported operating systems can be brought up; utilities within each operating system can then process the simulated paper-tape input followed by any desired additional utilities (such as calling up an assembler program).

- 2) Send PROGRAM.PAL. This command copies the contents of the PROGRAM.PAL file to the PTRIN.TXT file as described in 1) above, then a designated operating system is launched in a SIMH session. Any supported operating system can be specified for this operation. However, the same operating system is always used when executing this command.

Because the PROGRAM.PAL file is always copied to the PTRIN.TXT file when using the Send PROGRAM.PAL command, it is not necessary to close the editing session, which can stay active in a separate window. By using this method, it is possible to achieve rapid program file development; the editing session need only be closed when done with all development operations.

Note: To use multiple operating systems to process the same file, it is necessary to close the editing session as described above in 1). At that point, opening a new SIMH session to any operating system (which does not specify any editing or other modification of the PTRIN.TXT file) will pass the last edited file contents in the PTRIN.TXT file. This can be accomplished repeatedly since, in general, only closing editing sessions or send command execution will change the PTRIN.TXT file. This can be useful when comparing operating system environments such as OS/8 and P?S/8 when passed the same file or taking advantage of features unique to each operating system.

The underlying implementation of these two commands are two related batch files called Edit and Copy PROGRAM.PAL.BAT and Send PROGRAM.PAL.BAT stored in the SIMH Batch Files directory. These files (and the shortcuts/links to them generally placed in the Command directory when actively used) need to be adapted to create an equivalent set of batch files and shortcuts/links for each user-defined file; the subject file illustrated here is only provided as a working example. Note: Additional files may be provided in the Inactive Programs directory based on the works of other users.

The user can remove all such unused components as necessary to avoid clutter when adding in new user-defined components. Note: It is recommended that unneeded shortcuts/links be moved to the Inactive Programs directory already provided for this purpose; the underlying batch files require very little storage and might be useful for future user file development projects.

Note: The PROGRAM.PAL file, as provided in the PEPS package, is a dummy file intended for use by those who want to quickly develop a short program in PDP-8 Assembly Language (PAL). By foregoing a distinctive name, any user can quickly develop this file as far as deemed necessary. As required, the file can later be copied to another directory (and renamed as the user wishes) using Windows Explorer techniques.

Depending on requirements, future work can be initiated in a variety of ways. For slightly larger projects, it is recommended that the user deploy the Current_Program_Files directory technique as referenced above and described in further detail in Section 1.62 below. While this requires some slight additional setup effort on the part of the user, this is easily mastered by beginners after some minimal experience with the PEPS environment.

Advanced users can create custom command pairs patterned after the method used to develop PROGRAM.PAL while applying their own unique filenames. Using additional advanced techniques, files can be located somewhere within the recommended file structure group (PDP-8\Source Files) or on other windows directories/devices as deemed appropriate.

Appendix H documents the relevant parameters that must be set to implement any of these methods; not every project should necessarily be setup using the same overall method. Short-term projects using PROGRAM.PAL can be used instantly while named files residing in the designated current program files directory may describe a multiple file project. Only the most advanced projects may require resorting to the custom-named files method.

Each method has a slightly different set of tradeoffs; as user experience grows, it will become apparent which approach serves best for any programming project. In certain cases, working with source files created by others would contemplate little to no file editing once the proper file contents are confirmed. The custom command pairs method would generally be too tedious in this situation; copying all such files to the current program files directory would likely suffice.

1.50 Operating the PEPS from the Command directory

The Command directory arrangement (shown in Figure 3 below) includes all commands associated with the current release. This includes working commands provided as examples illustrating host editing of specified text files and the transmission of updated files into the SIMH environment running one of the supplied operating systems. Additional commands are provided to process various files returned from the SIMH environment as well as perform other functions as described below.

The specific details of the Command window are subject to user preference considerations with regard to approaching the Windows desktop. The commands (as shown in Figure 3 below) are configured according to certain Windows parameters and other factors currently in effect:

- a) Medium icons are in use; other available alternatives include both smaller and larger icons, full details and a larger tiled format; some of these alternatives will fully display the captions labeling all icons regardless of label character count (albeit likely occupying a larger portion of the overall Windows desktop).

Note: In this example, one of the icons was deliberately selected (thus the displayed color is darker) for the purpose of expanding the longer caption text. Due to the way Windows operates, when not selected, the caption text may become truncated unless one of the other view alternatives is used.

In general, most of the captions are completely shown even when not selected; the medium icons and supplied captions represent a reasonable compromise of readability and clutter avoidance.

- b) All command icons are positioned by asserting Align to Grid with the associated Auto Arrange option disabled. Settings to accomplish this are available within the window view particulars available in all supported windows versions (perhaps requiring a small patch to the desktop controls).

Note: Third-party patches for the affected systems are described in Appendix I.

- c) The grouping of icons in the Command window (as illustrated in Figure 3 below) is primarily by similarity of function. As necessary, empty positions are deliberately reserved for the purpose of future expansion. Additional commands will likely be added in future releases. Several examples of specific file editing and transmitting the updated file to the SIMH simulator are provided. Additionally, a series of commands is provided to implement generic file processing requiring no changes to the Command window.
- d) The general icon color scheme chosen includes specific colors for related functions as well as colors associated with two of the supported operating systems (OS/8 and P?S/8 configured without limitations). These color cues assist the user in selecting the desired function. Advanced users can modify any of these considerations; however, beginners are advised to work with the standard release as documented here.

Note: Icon organization within the Command window must be accomplished using windows commands to allow icon position to be determined manually; icon alignment is recommended for an orderly appearance. For windows releases past windows Vista, slight system modifications may be necessary. Appendix J contains information on utilities that may be required.

Undoubtedly, future releases of the PEPS package will contain numerous additional icons for a variety of reasons (such as variations of system configurations for the various supported operating systems). As the package evolves, various methods will be suggested for removing clutter in several different categories. Perhaps families of similar operating systems differing only in boot device specifics or handler configuration can be stored in an auxiliary window stored within the Command window. The user can place only the most frequently used configurations into the easiest accessible window positions.

Note: For the convenience of the user, the PEPS package includes an icon/shortcut to the Command directory, which is intended to be placed on the main desktop. By opening the Command directory, most PEPS features are immediately available (including easy access to specific auxiliary directories as needed).



Figure 3. PEPS Command Directory window

Advanced users are encouraged to create custom icons for new command shortcuts/links to be added to the Command directory; new icon files must be placed in the existing icon directory (*Links and Support Files*) prior to editing shortcut properties (generally using a copy of an existing shortcut/link as a template). This will greatly simplify the editing process; otherwise, windows acts in a presumptive manner which can be quite frustrating. Note: This is a severe limitation of windows that cannot (easily) be overcome.

By using the provided directory for all icon files, the path strings within the shortcut can be safely edited changing only the trailing characters in the path string (which is the icon filename) while maintaining the original leading portion of the path string. More information on creating custom shortcuts/links is available in Appendix H.

1.60 Description of individual commands within the Command window

The six rows of the command window shown in Figure 3 above are described below, each in left to right order. With few exceptions, each command is a shortcut/link to a corresponding batch command file stored in the SIMH Batch Files directory.

1.61 Supported Operating Systems

As shown in Figure 3 above, commands to launch various operating systems are located in the top row of the Command directory as follows:

- a) OS/8. This is generally OS/8 Version 3D (also known as OS/8 Version 3Q). In specific instances, system program components have been patched to correct known issues. Certain additional programs from newer OS/8 family releases are included that are generally improvements on the original releases. While there may be minor quirks or other performance trade-offs using these programs, this should be of no practical consequence when used in the PEPS environment.

Note: Some of the added programs merely fail to be usable on real-world hardware systems prior to the PDP-8/E; since the SIMH simulator supports all instructions from the PDP-8/E (KK8F) CPU, this is of no consequence. Users intending to transfer the system images contained in the PEPS package to real-world hardware are advised of the inherent limitations.

Certain non-DEC programs are also provided to perform additional tasks, such as the Kermit-12 utilities written by Charles Lasner. These programs are needed to process specific files obtained externally from other users usually by means of the Internet.

The included configuration of OS/8 boots to the RK8E/RK05, generally mounted on RK05 unit 0. By changing the configuration of mounted drives, it is possible to boot OS/8 on any RK05 drive 0-3 using supported techniques of the SIMH simulator or running short programs from another operating system.

Due to the manner in which the host windows environment passes files to the SIMH simulator, it is necessary to include the standard OS/8 PTR: handler to process paper-tape input unless P?S/8 is booted first.

When finished, P?S/8 will include the *PTUTIL* program that does not have the program defect described below. (Note: *PTUTIL* is the latest P?S/8 utility currently under development at the time of this writing.) After the PTR: input is correctly processed by P?S/8, the P?S/8 *OS8CON* utility can be used to pass the resultant file to an OS/8 device (such as DTA1:) available in the OS/8 configuration.

Because of a known quirk of the OS/8 PTR: handler, the very last character of the simulated PTR: character stream (contents of the *PTRIN* file in the *SIMH_Working_Files* directory) will be ignored; as such, any file obtained from the PTR: handler will be slightly truncated. If the passed file is an ASCII text file, the lost input will generally be a trailing <LF> character (which can be restored by passing the file through OS/8 *PIP* or various editing programs such as OS/8 *TECO*).

Certain programs which process <CR> characters and ignore <LF> characters will function correctly when reading these files despite the slight defect. Binary files (which generally have multiple leader/trailer characters) will generally not be affected by the PTR: handler quirk.

Additional bootable OS/8 systems can be added that can access other devices besides the RK8E/RK05 to support such as the RX8E/RX01 or boot to other devices such as the TD8E DECTape (or support alternate non-system handlers); it is the nature of OS/8 that many viable system configurations can be supported. Additional configuration files and program shortcut/links can be added to the PEPS package as desired.

- b) P?S/8. This is Version 8Z of P?S/8 configured for the TC01/TC08 DECTape with up to eight logical drives numbered 0 through 7. Over the years, various patches have been applied to P?S/8 Version 8Z to correct certain known bugs. An *Engineering Change Order (ECO)* log file is available to detail what has been modified since the original release of Version 8Z on 23-Feb-1988.

Note: Future support of P?S/8 within the PEPS environment will be upgraded with the release of P?S/8 Version 9A which is currently under development.

The *PTUTIL* and *BINCON* utilities will be added to Version 8Z when available, likely using the temporary system program execution method described within various P?S/8 user guides. These will be included in the future P?S/8 Version 9A release as normal system programs.

Note: Both OS/8 and P?S/8 are capable of booting any available operating systems within PEPS directly using simple commands or by running short boot-up programs which are provided in each system or are documented within Appendix J and can be trivially created manually.

In the case of the RK8E/RK05 cartridge disk device, this will allow boot-up to any of the four possible drives if a suitable cartridge file is mounted by proper inclusion in the relevant configuration file(s). All intersystem bootup procedures are documented in Appendix J.

- c) P?S/8 with the lineprinter disabled. In earlier releases of P?S/8, many programs automatically directed output to the device 66 lineprinter if it was available and online. To prevent this action, it was necessary to disable the lineprinter either by powering down or placing the device in an offline state.

As of the release of P?S/8 Version 8Z, most of these problems have been eliminated; command-line option switches exist in the affected programs that force output to the console terminal if desired. To allow testing of any remaining problems, P?S/8 may be started with the device 66 lineprinter logically disabled in the corresponding SIMH configuration file.

While fully supported, this system variation is of little interest to most PEPS users; once all known issues are resolved, this mode of operation will likely be withdrawn from future PEPS releases or moved to the Inactive Commands sub-directory.

Note: Future releases of the PEPS package will include additional operating systems such as P?S/8 configured for the TD8E/TU56 (possibly requiring the MR8E-C ROM), RK8E/RK05 or RX8E/RX01.

As the Command directory window becomes more complex, it may become necessary to include an internal folder for choice of operating system to avoid clutter. Advanced users may also require internal folders containing additional file development icons/links (other than a few currently present in the main folder) to further limit clutter. With experience, all users can determine which windows desktop particulars serve their needs in an efficient manner; consider the provided Command directory (as shown above in figure 3) as merely a starting point which can evolve as requirements change.

1.62 Generic file processing and related commands

Note: As shown above in Figure 3, all generic file processing and related commands are located in the second row of the Command directory.

The PEPS environment includes an internal folder for files currently under development which is referenced as the Current_Program_Files directory. This is in sharp contrast with files stored in a more general collection of external directories organized by user preference; a partial implementation of these directories is provided with each release of the PEPS package including certain example files that are accessible with explicit commands. This file structure should serve as a guide to additional user directory organization and file development.

Commands described in this section are available to interact with files in both the Current_Program_Files directory and the external directories using Windows Explorer techniques; most of these commands are geared towards operating on files within the Current_Program_Files directory.

- a) Edit Current Program. The current named file is opened in a new window using the designated editor, in this case Windows Notepad. The Current Program filename is set using the Set Current Program command as described in c) below.

Note: Appendix C contains information regarding conforming alternative editing programs.

Closing the editing program generally saves the updated state of the current named file; the file is then copied to the PTRIN.TXT file, the contents of which are used as the input to the PTR: device within the SIMH simulator environment. This allows any operating system to be launched with the current named file already set up to be read in when needed; however, it is not necessary to close the editing program:

If the current named file is saved using appropriate commands within the editor, the Send Current Program command described in b) below can be used to process the file in the designated operating system perhaps followed by other programs deemed necessary for development in both the simulated and host environments; as required, this process can be repeated. This is the fastest way to develop most PDP-8 programs taking advantage of the host environment file structure and certain relevant PEPS features described elsewhere in this document.

Note: All aspects of the Edit Current Program command are defined in the Edit Current Program.BAT file in the SIMH Batch Files directory.

- b) Send Current Program. The contents of the current named file are copied to the PTRIN.TXT file used as the input to the PTR: device within the SIMH environment. The SIMH simulator is then launched in a separate window, booting to the designated operating system. The Current Program filename is set using the Set Current Program command described in c) below.

Note: All aspects of the Send Current Program command are defined in the Send Current Program.BAT file in the SIMH Batch Files directory.

As released, PDS/8 with standard LPT: support is the designated operating system for use with this command. By making a small change to the Send Current Program.BAT file, any available operating system can be chosen. Several operating system choices are already provided within the batch file; all inactive choices should be preceded with a *REM* statement to simplify further modifications in the future.

- c) Set Current Program. The Current Program filename (as referenced in a) and b) above) is set using this command.

Note: The Current Program filename text string is encoded and permanently stored in the installed PEPS package within a key parameter file. This command is the only method available to update the filename; this applies even if the entire PEPS package is repackaged for transport to another windows system.

As shown in Figure 4 below, the default Current Program file name is presently *HELLOWORLD.PAL*. This command allows the filename to be changed to any desired combination of alphanumeric characters in upper or lower case; while embedded space characters are not allowed, several additional characters (such as *[* and *]*) are permitted.

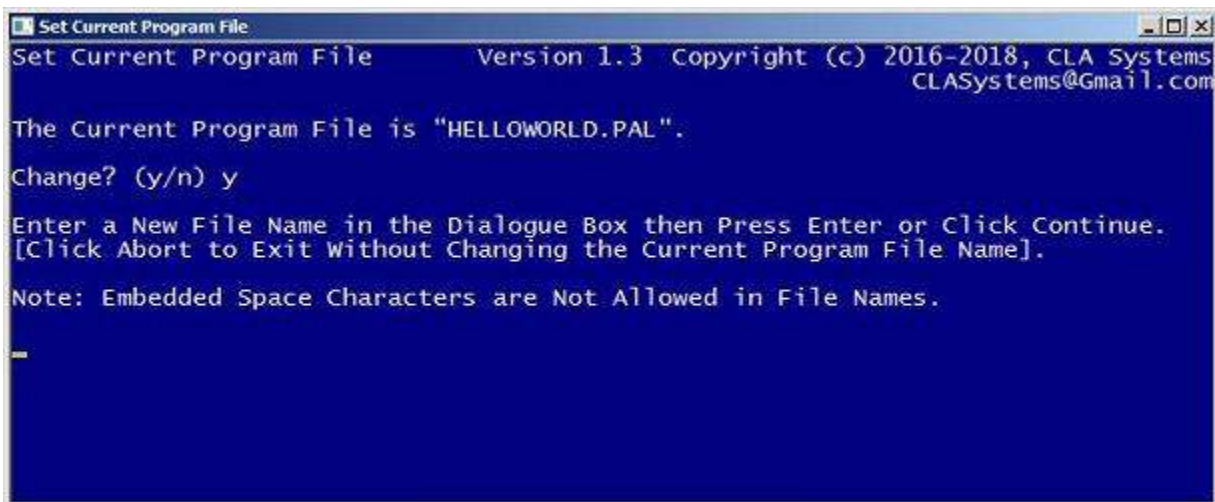


Figure 4. Set Current Program File Screen

As shown in Figure 5 below, the Current File name is being changed to *MYPROGRAM.PAL* by manual entry into the associated dialogue box launched by the command when a change is requested. The user can abort the process by clicking on the **Abort** button (or the **X** in the upper right corner) or confirm the requested new file change by clicking on the **Continue** button (or pressing the **Enter** key on the keyboard).

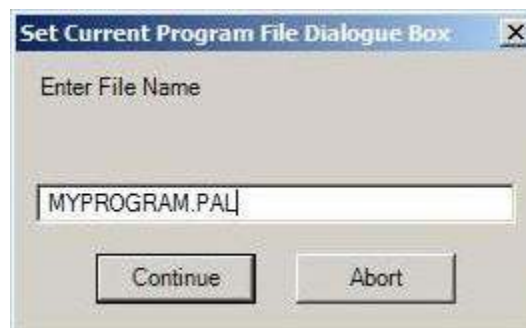


Figure 5. Set Current Program File Dialogue Box

Once the filename request is processed, the new value of the Current Program filename applies. As shown in Figure 6 below, if the new Current Program file does not exist, a new file will automatically be created in the Current_Program_Files directory. When the command exits, the Current Program filename will permanently be changed to *MYPROGRAM.PAL* unless and until another Set Current Program command is executed.

Note: All aspects of the Set Current Program command are defined in the Set Current Program.BAT file in the SIMH Batch Files directory.

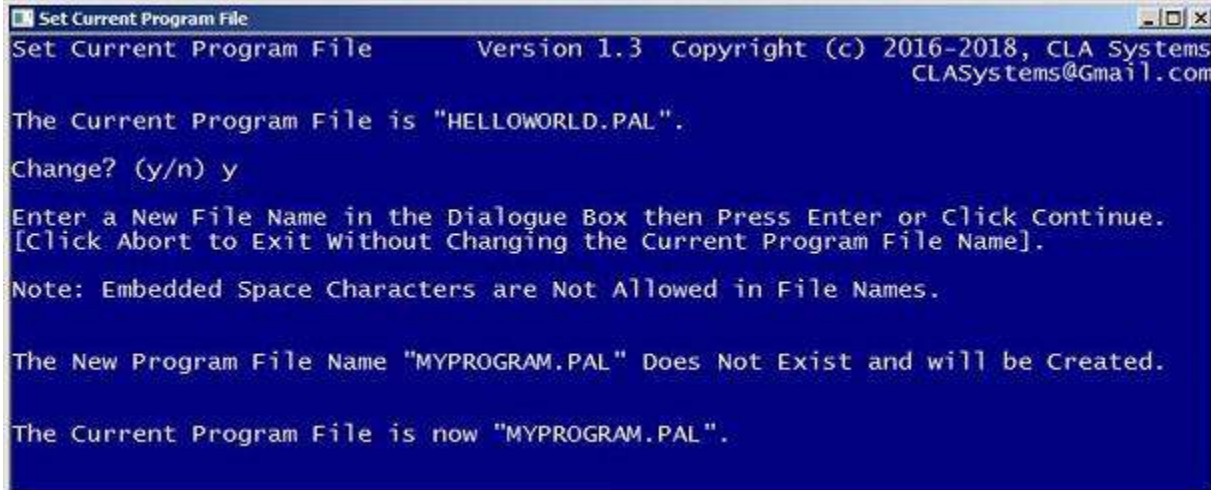


Figure 6. Set Current Program File Screen after filename change

- d) Explore Current Program Files. The Current_Program_Files directory is open in a separate window. Files can be transferred in or out of this directory from other folders as required using standard Windows Explorer techniques.

Note: this command is often used in conjunction with the Explore Additional Program Files command as described in f) below.

- e) List Current Program Files. As shown in Figure 7 below, a listing file of the Current_Program_Files directory is created then opened in a separate window using the standard windows Notepad utility. This file may be modified as necessary and saved or printed if desired.

Note: The Set Current Program dialogue box as described above in c) can be filled by applying copy and paste techniques to this window to aid in minimizing manual entry where applicable. Existing filename characters can be used to (partially) set the Current Program filename without having to manually type the entire filename. For example, the existing file name *PROGRAM.PAL* can be copied and then modified into *PROGRAM1.PAL* or *PROGRAM2.PAL*, etc.

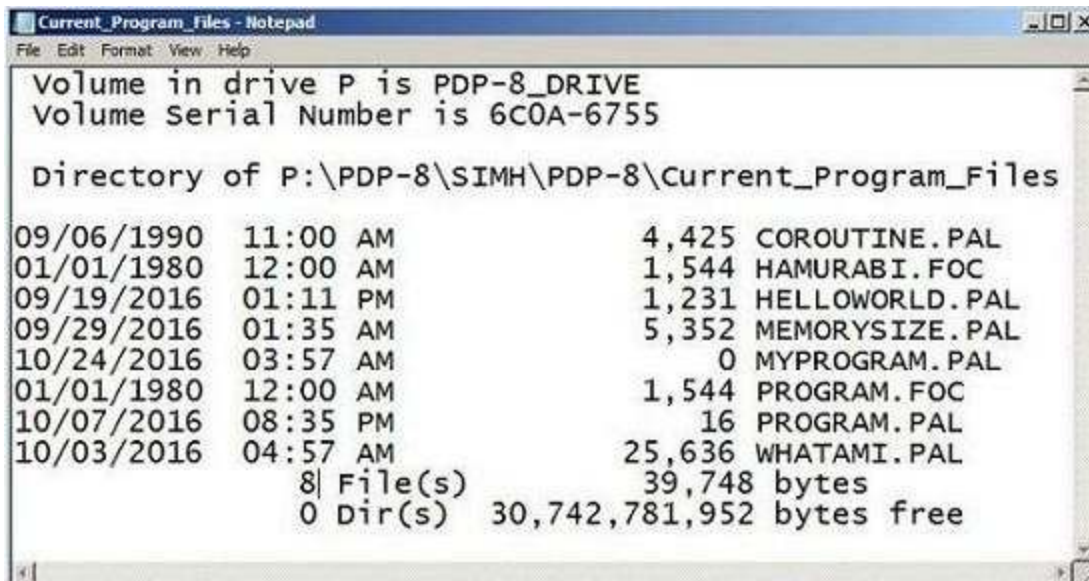


Figure 7. Current_Program_Files Directory Listing

- f) Explore Additional Program Files. A separate window is opened to the *PDP-8\Source Files* directory. It is recommended that all files oriented towards PDP-8 projects be organized into appropriate sub-folders of this directory. Files stored in any of these directories can be interchanged with those of the Current_Program_Files directory using windows Explorer techniques.

1.63 Custom commands for individual file development

Note: As shown in Figure 3 above, all custom commands for individual file development are located in the third row of the Command directory window.

Some of the command examples supplied in this section are associated with the OS/8 operating system while others are associated with the P?S/8 operating system. While this generally pertains to which operating system is chosen when sending the file via the PTRIN.TXT mechanism, the corresponding command icons/links are color-coded for consistency. Black icons are selected for files to be developed using OS/8 while blue icons are selected for files to be developed using P?S/8.

Advanced users can derive commands for development of additional files using any desired color scheme (or alternate icon set). Appendix E provides information for advanced users to aid in creating new batch files and corresponding command shortcut/icon entries. The provided files described in this section can be used as templates to create custom additions to the standard installation.

Note: *COROUTINE.PAL* is an example of how to implement efficient coroutines taking good advantage of the PDP-8 instruction set. This is a code fragment taken from the main *Kermit-12* program written by Charles Lasner as part of the overall Kermit-12 project.

- a) Edit *COROUTINE.PAL*. The text file \PDP-8\Source Files\Generic Source Files\COROUTINE.PAL is opened in a new window using the designated editor, in this case Windows Notepad.

Note: Appendix C contains information regarding conforming alternative editing programs.

Closing the editing program generally saves the updated state of the *COROUTINE.PAL* file; the file is then copied to the *PTRIN.TXT* file, the contents of which are used as the input to the PTR: device within the SIMH simulator environment. This allows any operating system to be launched with the *COROUTINE.PAL* file already set up to be read in when needed. However, it is not necessary to close the editing program:

If the *COROUTINE.PAL* file is saved using appropriate commands within the editor, the Send *COROUTINE.PAL* command described in b) below can be used to process the file in the designated operating system perhaps followed by other programs deemed necessary for development in both the simulated and host environments; as required, this process can be repeated. This is the fastest way to develop most PDP-8 programs taking advantage of the host environment file structure and certain relevant PEPS features described elsewhere in this document.

Note: All particulars of the Edit *COROUTINE.PAL* command are defined within the Edit and Copy *COROUTINE.PAL.BAT* file in the SIMH Batch Files directory.

- b) Send *COROUTINE.PAL*. The contents of the \PDP-8\Source Files\Generic Source Files\COROUTINE.PAL file are copied to the *PTRIN.TXT* file used for input to the PTR: device within the SIMH environment. The SIMH simulator is then launched in a separate window, booting to the designated operating system.

As released, OS/8 is the designated operating system for use with this command. By making a small change to the Send *COROUTINE.PAL.BAT* file, any available operating system can be chosen. Several operating system choices are already provided within the batch file; all inactive choices should be preceded with a REM statement to simplify further modifications in the future.

Note: All aspects of the Send *COROUTINE.PAL* command are defined in the Send *COROUTINE.PAL.BAT* file in the SIMH Batch Files directory.

Since both a) and b) (as described above) refer to a file developed within the OS/8 environment, the associated icons for these commands are black color.

- c) Edit MEMORYSIZE.PAL. The text file `\PDP-8\Source Files\Generic\Source Files\MEMORYSIZE.PAL` is opened in a new window using the designated editor, in this case windows Notepad.

Note: Appendix C contains information regarding conforming alternative editing programs.

Closing the editing program generally saves the updated state of the MEMORYSIZE.PAL file; the file is then copied to the PTRIN.TXT file, the contents of which are used as the input to the PTR: device within the SIMH simulator environment. This allows any operating system to be launched with the MEMORYSIZE.PAL file already set up to be read in when needed. However, it is not necessary to close the editing program:

If the MEMORYSIZE.PAL file is saved using appropriate commands within the editor, the Send MEMORYSIZE.PAL command described in d) below can be used to process the file in the designated operating system perhaps followed by other programs deemed necessary for development in both the simulated and host environments; as required, this process can be repeated. This is the fastest way to develop most PDP-8 programs taking advantage of the host environment file structure and certain relevant PEPS features described elsewhere in this document.

Note: All particulars of the Edit MEMORYSIZE.PAL command are defined within the Edit and Copy MEMORYSIZE.PAL.BAT file in the SIMH Batch Files directory.

- d) Send MEMORYSIZE.PAL. The contents of the `\PDP-8\Source Files\Generic Source Files\MEMORYSIZE.PAL` file are copied to the PTRIN.TXT file used for input to the PTR: device within the SIMH environment. The SIMH simulator is then launched in a separate window, booting to the designated operating system.

As released, OS/8 is the designated operating system for use with this command. By making a small change to the Send MEMORYSIZE.PAL.BAT file, any available operating system can be chosen. Several operating system choices are already provided within the batch file; all inactive choices should be preceded with a REM statement to simplify further modifications in the future.

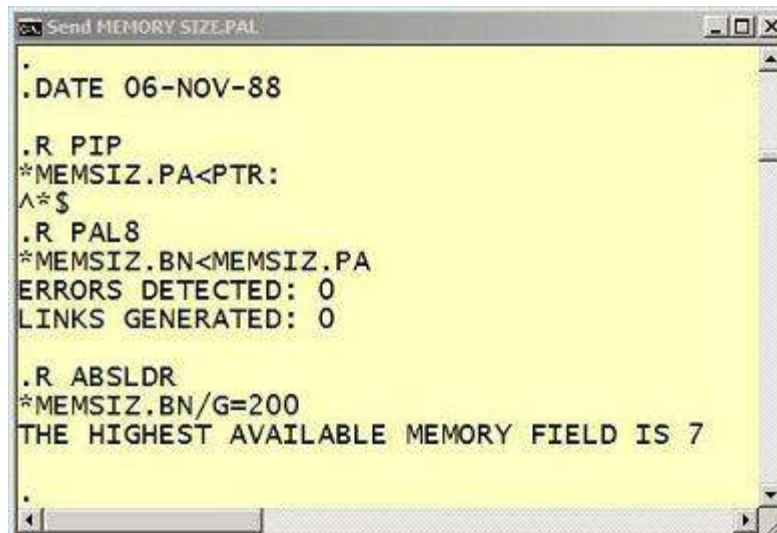
Note: All aspects of the Send MEMORYSIZE.PAL command are defined in the Send COROUTINE.PAL.BAT file in the SIMH Batch Files directory.

Note: Since both c) and d) (as described above) refer to a file developed within the OS/8 environment, the associated icons for these commands are black color.

An OS/8-specific SIMH session using the contents of MEMORYSIZE.PAL passed as the input to the PTR: device is shown in Figure 8 below. The various steps taken are documented as follows:

- i) The OS/8 date is set to 06-Nov-88. This date was chosen because it allows the day of the week to correspond to that which applies on 06-Nov-2016. OS/8 dates are not preserved across SIMH bootup sessions.

Note: OS/8 only allows dates in the range of 01-Jan-1970 through 31-Dec-2001 with eight-year group anomalies for all dates past 31-Dec-1977.
- ii) The OS/8 *PIP.SV* utility is executed to create the file *MEMSIZ.PA* from the paper-tape input stream setup when the SIMH session was initiated.
- iii) The OS/8 *PAL8.SV* utility is executed to assemble the contents of the *MEMSIZ.PA* file into the *MEMSIZ.BN* binary file.
- iv) The OS/8 *ABSLDR.SV* utility is executed to load the *MEMSIZ.BN* binary file into memory; by specifying */G=200*, the program is automatically started.
- v) The console output produced by program execution is shown in Figure 8 below. Since the SIMH simulator is configured to provide a 32K memory environment, there are eight memory fields available numbered 0 through 7; the highest memory field is correctly indicated as 7.



```
.
.DATE 06-NOV-88

.R PIP
*MEMSIZ.PA<PTR:
^*$
.R PAL8
*MEMSIZ.BN<MEMSIZ.PA
ERRORS DETECTED: 0
LINKS GENERATED: 0

.R ABSLDR
*MEMSIZ.BN/G=200
THE HIGHEST AVAILABLE MEMORY FIELD IS 7
.
```

Figure 8. OS/8 SIMH session using MEMORYSIZE.PAL

- e) Edit WHATAMI.PAL. The text file `\PDP-8\Source Files\Generic\Source Files\WHATAMI.PAL` is opened in a new window using windows Notepad (the designated text editing program). All particulars of this command are defined within the *Edit and Copy WHATAMI.PAL.BAT* file in the *SIMH Batch Files* directory. As explained above with regard to the *Edit Current Program* command, Appendix C contains information regarding conforming alternative editing programs.

Closing the editing program generally saves changes made to the *WHATAMI.PAL* file; the contents of the file are copied to the *PTRIN.TXT* file used as input to the PTR: device during the next SIMH simulator session. This allows any available operating system to be launched with the contents of *WHATAMI.PAL* already setup within the *PTRIN.TXT* file for capture within the selected PDP-8 operating system.

For most editing situations it is not necessary to close the editing program each time file updates are made. Merely by saving the file using an appropriate editor command, the *Send WHATAMI.PAL* command (as described in f) below) can be used to initiate a SIMH session that can produce appropriate results leading to another file edit cycle, etc. This is the fastest method to develop most PDP-8 programs taking advantage of features of the host windows environment.

- f) Send WHATAMI.PAL. The contents of the `\PDP-8\Source Files\Generic Source Files\WHATAMI.PAL` file are copied to the *PTRIN.TXT* file used for input to the PTR: device within the SIMH environment. The SIMH simulator is then launched in a separate window, booting to the designated operating system (as defined in the corresponding underlying batch file associated with this command [*Send WHATAMI.PAL.BAT*]). Note: while P?S/8 is the default operating system for this function, a minor edit to this batch file can be made to cause a bootup to OS/8 or other available operating system; this option exists in all analogous batch files to provide free choice of operating system when sending files via the PTR: mechanism.

Note: Since both e) and f) (as described above) refer to a file developed within the P?S/8 environment, the associated icons for these commands are blue color.

A P?S/8-specific SIMH session using the contents of *WHATAMI.PAL* passed as the input to the PTR: device is shown in Figure 9 below. The various steps taken are documented as follows:

- i) The P?S/8 date was set to 06-Nov-2016 in a prior SIMH session. Unlike OS/8, P?S/8 retains the current date permanently until changed.

Note: Date handling in P?S/8 Version 8Z can be extended through 31-Dec-2099 by applying maintenance patches roughly every eight years. Future releases will extend date handling through 31-Dec-2411 without requiring any maintenance procedures.

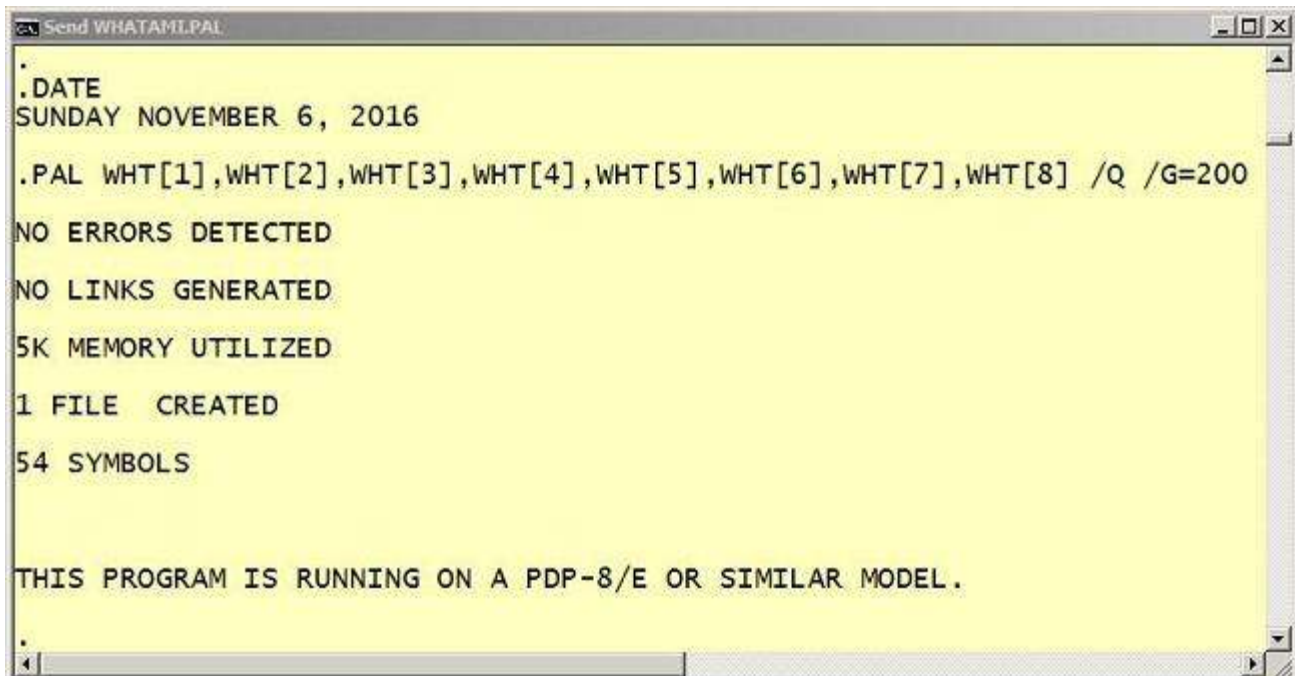
- ii) The *WHATAMI.PAL* file was processed into the eight *Tiny File System* (TFS) files *WHT[1]*, *WHT[2]*, *WHT[3]*, *WHT[4]*, *WHT[5]*, *WHT[6]*, *WHT[7]* and *WHT[8]* from the paper-tape input stream setup in a prior SIMH session.

Note: P?S/8 also supports extended-length files. If this method had been chosen, the program could have been accessed by using a single file name such as *WHTAMI*.

The eight TFS files are passed to the P?S/8 *PAL* assembler program; command-line option switches initiate the assembly process followed by an automatic chain to the P/S/8 system program *BIN* which performs binary file loading when used in this manner. The default binary file % is used as the output target of the assembly and then as the binary input file passed to P?S/8 *BIN*. Since a starting address is passed using */G=200*, the program starts automatically after loading.

Note: The assembler statistics indicate there were no assembly errors. Had there been any errors, the automatic chain to P?S/8 *BIN* would have been aborted.

- iii) The *WHATAMI.PAL* program is designed to detect the specific PDP-8 hardware model the program is running on. Since the SIMH simulator is configured to support the KK8F CPU, the program indicates it is running on a PDP-8/E or similar model.



```
.
.DATE
SUNDAY NOVEMBER 6, 2016

.PAL WHT[1],WHT[2],WHT[3],WHT[4],WHT[5],WHT[6],WHT[7],WHT[8] /Q /G=200

NO ERRORS DETECTED
NO LINKS GENERATED
5K MEMORY UTILIZED
1 FILE  CREATED
54 SYMBOLS

THIS PROGRAM IS RUNNING ON A PDP-8/E OR SIMILAR MODEL.
.
```

Figure 9. P?S/8 SIMH session using *WHATAMI.PAL*

- g) Edit FOCAL Program. The text file `\PDP-8\SIMH\PDP-8\Current_Program_Files\PROGRAM.FOC` is opened in a new window using windows Notepad (the designated text editing program). All particulars of this command are defined within the *Edit and Copy FOCAL Program.BAT* file in the *SIMH Batch Files* directory. As explained above with regard to the *Edit Current Program* command, Appendix C contains information regarding conforming alternative editing programs.

Closing the editing program generally saves changes made to the *PROGRAM.FOC* file; the contents of the file are copied to the *PTRIN.TXT* file used as input to the PTR: device during the next SIMH simulator session. This allows any available operating system to be launched with the contents of *PROGRAM.FOC* already setup within the *PTRIN.TXT* file for capture within the selected PDP-8 operating system.

For most editing situations it is not necessary to close the editing program each time file updates are made. Merely by saving the file using an appropriate editor command, the *Send PROGRAM.FOC* command (as described in h) below) can be used to initiate a SIMH session that can produce appropriate results leading to another file edit cycle, etc. This is the fastest method to develop most FOCAL (and other PDP-8 programs) taking advantage of features of the host windows environment.

- h) Send FOCAL Program. The contents of the `\PDP-8\SIMH\PDP-8\Current_Program_Files\PROGRAM.FOC` file are copied to the *PTRIN.TXT* file used for input to the PTR: device within the SIMH environment. The SIMH simulator is then launched in a separate window, booting to the designated operating system (as defined in the corresponding underlying batch file associated with this command [*Send FOCAL Program.BAT*]). Note: while P?S/8 is the default operating system for this function, a minor edit to this batch file can be made to cause a bootup to OS/8 or other available operating system; this option exists in all analogous batch files to provide free choice of operating system when sending files via the PTR: mechanism.

Note: Since both g) and h) (as described above) refer to a file developed within the P?S/8 environment, the associated icons for these commands are blue color; however, since these commands are oriented towards FOCAL programs, a somewhat lighter shade of blue was chosen.

The default contents of the *PROGRAM.FOC* file is an accurate copy of the historic DEC *King of Sumeria* game known as *HAMURABI*. This file is also included in the `\PDP-8\Source Files\PQS8 FOCAL - Related Files` directory as *HAMURABI.FOC*.

A P?S/8-specific SIMH session using the contents of *PROGRAM.FOC* passed as the input to the PTR: device is shown in Figure 10 below. The various steps taken are documented as follows:

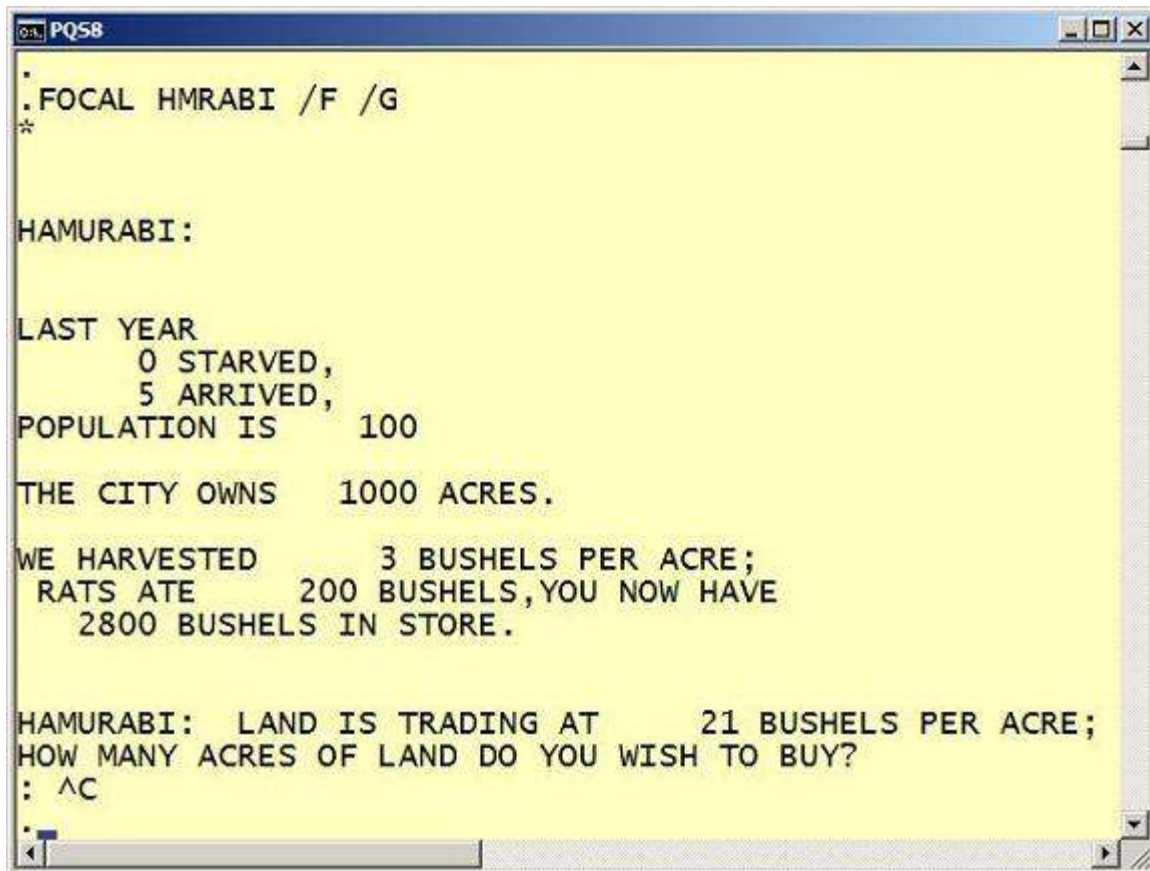
- i) As described above, the paper-tape input stream setup in a prior SIMH session is the contents of *PROGRAM.FOC* which in turn is a copy of *HAMURABI.FOC*. As is usually the case with most short FOCAL programs, the entire program was converted into a single TFS file which was named *HMRABI*.
- ii) The *HMRABI* file is then passed to P?S/8 *FOCAL* which is a binary compatible superset of the original DEC FOCAL, 1969 release. The contents of passed binary and text files are used to implement custom binary patches to FOCAL and pass FOCAL programs and data respectively. In this instance, no binary patches are required.

Various command-line switches (both present and deliberately omitted), control the precise way FOCAL operates. In this instance, the Sine and Arctangent option groups are not loaded to maximize the memory space allocated to the program, variables and pushdown stack.

The source file contents do not echo on the console device during the loading process; this operation can be enabled by changing the command-line options. Automatic program startup (after displaying the FOCAL command prompt character) is also initiated by the command-line options selected.

- iii) The running program presents various options to the local town manager (*HAMURABI*) requiring some strategy as how best to proceed allocating the town's resources. Execution was stopped by the use of the Control-C character to abort the FOCAL program.
- iv) Users very familiar with this version of the *HAMURABI* game, as documented in various DEC handbooks, may notice subtle differences in the program output as shown in Figure 10 below. The original FOCAL, 1969 supplies a leading = character on all numerical output; the output format would include an extra leading space character which tends to make the numeric values slightly harder to read.

It is the general consensus of the PDP-8 FOCAL user community at large these features are quirks which should be suppressed; as such, P?S/8 FOCAL removes the affected characters by default. Command-line options are available to restore either or both historic features if desired.



```
PDS/8
*
.FOCAL HMRABI /F /G
*

HAMURABI:

LAST YEAR
    0 STARVED,
    5 ARRIVED,
POPULATION IS    100

THE CITY OWNS    1000 ACRES.

WE HARVESTED      3 BUSHEL PER ACRE;
RATS ATE          200 BUSHEL, YOU NOW HAVE
    2800 BUSHEL IN STORE.

HAMURABI: LAND IS TRADING AT    21 BUSHEL PER ACRE;
HOW MANY ACRES OF LAND DO YOU WISH TO BUY?
: ^C
```

Figure 10. PDS/8 FOCAL SIMH session using PROGRAM.FOC

1.64 Utility commands for paper-tape and lineprinter devices

Note: As shown above in Figure 3, all special-purpose utility commands are located in the fourth row of the Command directory window.

These commands are permanent components of the PEPS package targeting files used to communicate between the host windows environment and the SIMH simulated environment; the files are located in the `\PDP-8\SIMH_Working_Files` directory. The commands are used to implement various operations on these files as needed regardless of which supported operating system is being used at the time.

File descriptions are as follows:

- 1) *LPTOUT.TXT* - This file is used to capture the output directed to the device 66 lineprinter during any SIMH session. Various commands are implemented to manage the contents of this file.

By convention, the contents of the *LPTOUT.TXT* are presumed to be printable text generally formatted for use with real-word lineprinters; various provided utilities can post-process this file for additional purposes.

- 2) *PTPOUT.TXT* - This file is used to capture the output directed to the high-speed punch (PTP:) device. While the file extension used might imply printable text contents, the actual file contents could be non-printable binary coding depending on specific circumstances. Actual text contents can be opened in any designated editing program capable of processing ASCII text characters. Windows Notepad is the default program used throughout the PEPS package for such editing/display purposes. On a case by case basis, advanced users can modify the underlying command files to use an alternative text editing program if desired.
- 3) *PTRIN.TXT* - This file is used to transmit input data to the high-speed reader (PTR:) device. As described above in 2), the file extension might imply printable text contents. However, the actual file contents could be binary data such as a standard PDP-8 binary file in *BIN*^[7] format. In general, the user determines the specific contents of this file as well as the set of utilities within the designated current operating system that will process the file contents.

The various commands described in this section manage nearly all situations related to these key files. In certain rare instances, it may become necessary to use one of the miscellaneous commands (described in section 1.65 below) to accomplish certain additional tasks using Windows Explorer. This would generally concern transfer of files obtained via the Internet or other external resources. Appendix Q contains more information on optional commands to be included in this section or currently under development. If specific utilities are proposed and implemented, command links will be added to this section in future releases.

The commands available are as follows:

- a) Clear LPT Output. The *LPTOUT.TXT* file is emptied. This command is needed to ensure there are no artifact characters in the file resulting from activity in previous SIMH sessions. If the purpose of the next SIMH session is to produce a listing file directed to the device 66 LPT: device, this command must be used before initiating the SIMH session.
- b) Strip LPT Output. The *LPTOUT.TXT* file can contain artifacts created by imperfect operation of various output utilities in the supported operating systems. Left as created by the issuing programs, the file may be compromised in the Windows environment, especially if post-processing will be applied.

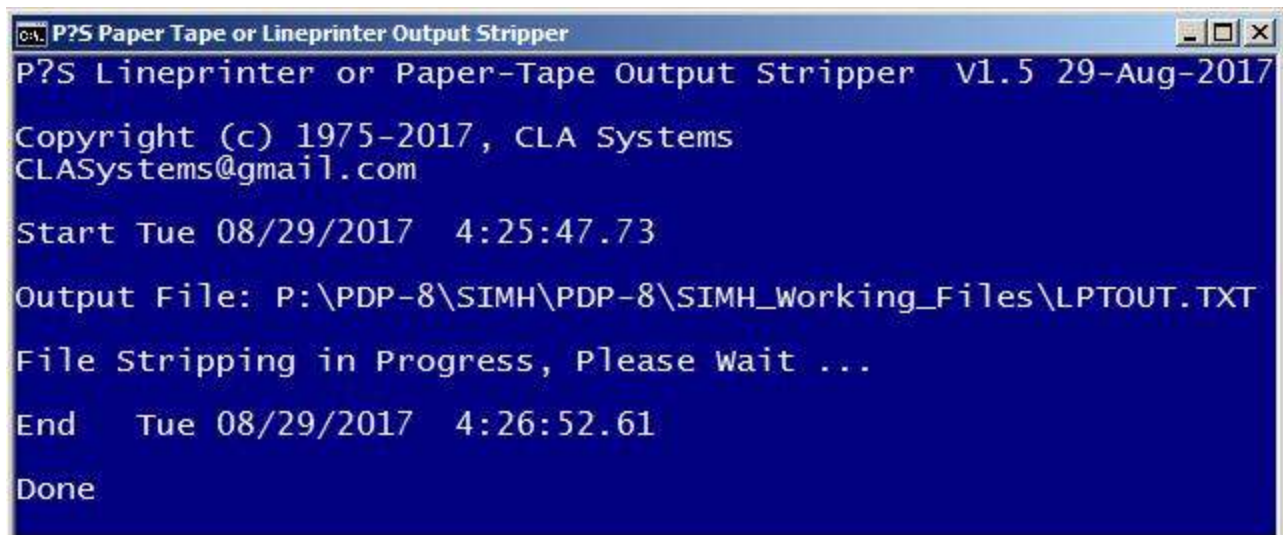
Note: Some of the problems with undesired *LPTOUT.TXT* file contents arise from necessary implementation restrictions within the original real-world PDP-8 environment; certain utilities will time out the LPT: device and then fall back on the console printer should the LPT: not be available. In some instances, test characters used would have been ignored in the sense that no artifacts would appear on the printed page of a real-world system lineprinter; however, the SIMH simulator captures all characters sent to the LPT: device.

As such, it is generally necessary to remove these unwanted characters from the *LPTOUT.TXT* file. In certain situations, the artifact characters, while present, are truly unfathomable or only work with specific printer models yet were never discovered during the economic lifetime of these operating systems. In the case of certain P?S/8 lineprinter quirks, the original development environment often lacked access to a lineprinter and the program merely implemented the best guesswork of the programmer. Some of these issues result in wasting a sheet of lineprinter paper; the intended output starts on the next page at the top of form point.

To solve all of the problems discussed above (and arcane additional issues not documented here), the PEPS package includes the *STRIP.TEC* macro designed to work with *TECOC*, a freeware implementation of DEC standard *TECO* for windows. Once a file is processed by *STRIP.TEC*, all such undesired output is eliminated; the resultant file is now ready for further processing such as input to a word-processor for the purpose of printing the file in windows. In some cases, post-processing might include conversion to PDF format for distribution to a wider audience or file archiving.

Running the *STRIP.TEC* macro on typical text files is a time-consuming process; the actual execution time depends on the windows system processing power, *LPTOUT.TXT* file length and other considerations. As shown below in Figure 11, a separate window is created to display the stripping process; the starting and ending times are displayed during the program execution to aid the user in determining future processing time should the file (or a subsequent version after a source file edit) require additional stripping.

while the example shown below in Figure 11 took just under 65 seconds to complete, it is not uncommon for very long assembly listings to take over five minutes for conversion into a properly formatted file when executed on many windows systems.



```
P?S Paper Tape or Lineprinter Output Stripper
P?S Lineprinter or Paper-Tape Output Stripper  V1.5 29-Aug-2017
Copyright (c) 1975-2017, CLA Systems
CLASystems@gmail.com

Start Tue 08/29/2017  4:25:47.73
Output File: P:\PDP-8\SIMH\PDP-8\SIMH_Working_Files\LPTOUT.TXT
File Stripping in Progress, Please Wait ...
End   Tue 08/29/2017  4:26:52.61
Done
```

Figure 11. LPTOUT.TXT file stripping session

- c) Open LPT Output. The *LPTOUT.TXT* file is opened in the Windows Notepad editor. This allows immediate display of the consequences of processing such as assembling a source file currently under development.

Note: While there is generally no need to modify the *LPTOUT.TXT* file, editing operations are freely allowed; the file may be printed if desired. In many circumstances, the font size must be reduced to obtain a complete display on the Windows desktop without resorting to horizontal scrolling. This can occur because language processing utilities add additional content at the left margin of each source line.

While alternative text editing programs may be useful for typical editing purposes, most other qualifying editing utilities may either cause all open text files to share a common window (by selecting one of a set of multiple tabs) or not support multiple font sizes in a single session. Unless the alternative program can support these additional features, it is recommended that Windows Notepad be used.

- d) Clear PTP Output. The *PTPOUT.TXT* file is emptied. This command is needed to ensure there are no artifact characters in the file resulting from activity in previous SIMH sessions. If the purpose of the next SIMH session is to send data to the PTP: device, this command must be used before initiating the SIMH session.
- e) Strip PTP Output. The *PTPOUT.TXT* file is stripped of unwanted artifacts; other than the change of target file from *LPTOUT.TXT* to *PTPOUT.TXT*, the entire process as described above in b) applies to this command.

It has been occasionally observed that the *PTPOUT.TXT* file will contain a trailing form-feed character after the file has been stripped of all other artifacts. The root cause of this minor issue has been determined to be an obscure quirk of the OS/8 PTP: handler due to an intrinsic design problem within the handler. Notably, if the identical file is processed by the OS/8 LPT: handler (into the *LPTOUT.TXT* file) the problem doesn't occur. Should this situation be problematic, the character removal can be accomplished manually using the *Open PTP Output* command (as described below in f); alternatively, the entire stripping process can be repeated to eliminate the unwanted artifact.

Due to the manner in which the OS/8 PTP: handler functions, while the form-feed character is the last such character in the original file, it is not actually the sentinel character of the file, generally surrounded by sequences of unwanted characters requiring removal at a different stage of the stripping process. Future releases of the *STRIP.TEC* macro may be able to remove this defect in more instances than as at present. (As of the current release of the stripping utility, this artifact is already being removed in many cases, albeit not in every case.)

- f) Open PTP Output. The *PTPOUT.TXT* file is opened in the standard Windows Notepad editor. This allows immediate display of the consequences of processing such as assembling the source file currently under development.

Other than the change of target file from *LPTOUT.TXT* to *PTPOUT.TXT*, the entire process as described above in c) applies to this command.

- g) Swap PTR and PTP Files. The contents of the *PTRIN.TXT* (PTR:) and *PTPOUT.TXT* (PTP:) files are interchanged. This provides file interchange useful in specific circumstances.

For example, binary files created in P?S/8 can be made available for OS/8 usage (or vice-versa) using this command. This allows OS/8 users to take advantage of the unique features of P?S/8 *PAL* which supports additional functionality beyond OS/8 *PAL8* while maintaining total *PAL* language compatibility.

With the availability of the P?S/8 *PTUTIL* program, text files may be directly interchanged between these operating systems without involving the host windows environment. Note: Other methods are available for file interchange between these two operating systems. Appendix V describes a variety of methods, some of which are currently under development.

- h) Swap PTP and LPT Files. The contents of the *PTPOUT.TXT* (PTP:) and *LPTOUT.TXT* (LPT:) files are interchanged. This provides file interchange useful in specific circumstances.

For example, data created on the paper-tape punch may be directed to the device 66 LPT: output file for further processing such as creation of a document in PDF format.

1.65 Miscellaneous commands

Note: As shown above in Figure 3, all miscellaneous commands are located in the fifth row of the Command directory window.

- a) SIMH Working Files. The *SIMH_Working_Files* directory is opened in a separate window. Using Windows Explorer techniques, various files may be accessed for such purposes as file interchange with external sources.

This folder is the working directory for device image files. By copying device image files here (and editing the SIMH configuration files) the contents of device image files can be made active in the SIMH environment. Appendix Y details the important files located in this directory.

Note: Normal activity of the various components of the PEPS package will tend to produce temporary files in this directory; in extreme cases, it may be necessary to manually remove temporary files. However, the majority of temporary filenames are internally recycled, largely avoiding any maintenance requirements.

- b) SIMH Config Files. The *SIMH Configuration Files* directory is opened in a separate window. Within this directory are the configuration files used with each variant operating system used with the SIMH simulator. Changes made to these files can greatly affect the way the corresponding operating systems function.
- c) Inactive Programs. The *Inactive Programs* directory is opened in a separate window. Generally, icon/link files are stored here for programming projects not currently under development. Several of these files are provided by advanced users of the PEPS package for the purpose of developing their own PDP-8 programs. All of the files are useful as templates for creating additional personal program development commands.

Note: The provided files in the *Inactive Programs* directory will not be actionable without the underlying support files. In some cases, the referenced source files are proprietary and are not available with the standard release of the PEPS package. However, the mechanism to exercise the icon/link files should generally be functional due to dummy files provided in lieu of the original source files.

- d) Convert LPT to PDF. The contents of the *LPTOUT.TXT* (LPT:) file is converted to the file *LPTOUT.PDF* in the *SIMH_Working_Files* directory using the *text2pdf* freeware program from VeryPDF.

The output is presented in 11" x 17" wide lineprinter paper format with a white background. Any conforming windows font may be used; the default distribution font is Consolas 10. Font particulars can be changed by modification of the *TEXT2PDF.BAT* program located in the *SIMH_Uilities* directory.

Note: Not all windows-compatible fonts are fully compatible with the VeryPDF program; advanced users may consider carefully attempting to use alternate fonts.

The key VeryPDF utility program file is not included in the standard release of the PEPS package for legal reasons. PEPS users may obtain the file from the author(s) of VeryPDF. Appendix Z provides further information on obtaining this file and how to add it to a working copy of the PEPS package.

Most PDF file readers can adjust the display size for easy viewing and output to printers of varying sizes. It is strongly recommended that Adobe Reader version 11.x be installed on the host windows system. Appendix F contains more information regarding recommended windows add-on utilities and recommended settings/patches.

- e) Convert LPT to PDF [Greenbar]. The contents of the *LPTOUT.TXT* (LPT:) file is converted to the file *LPTOUT.PDF* in the *SIMH_Working_Files* directory using the *text2pdf* shareware program from DaVince Tools.

The output is presented in 11" x 8.5" landscape compressed lineprinter paper format with a greenbar background. Output font is permanently set to Courier Bold 9.

Key DaVince Tools utility program files are not included in the standard release of the PEPS package for legal reasons. PEPS users may obtain the files from the author(s) of DaVince Tools. Appendix Z provides further information on obtaining these files and how to add them to a working copy of the PEPS package.

Most PDF file readers can adjust the display size and orientation for easy viewing and output to printers of varying sizes. It is strongly recommended that Adobe Reader Version 11.x be installed on the host windows system. Appendix F contains more information regarding recommended windows add-on utilities and recommended settings/patches.

- f) Help Documents. The *SIMH Documents* directory is opened in a separate window, allowing access to a variety of documentation files on various PEPS-related subjects. Since most of the files are in .PDF format, a suitable program such as *Adobe Reader* must be installed. Appendix U provides additional information on the available documents.
- g) Open Log File. The SIMH simulator logging file is opened in the standard windows Notepad editor. It may be useful to excise and edit a section of the logging file to document some issue with SIMH operation.
- h) Setup Log File. As shown in Figure 12 below, SIMH session logging can be enabled or disabled using this command. The session logging file can also be cleared if desired.

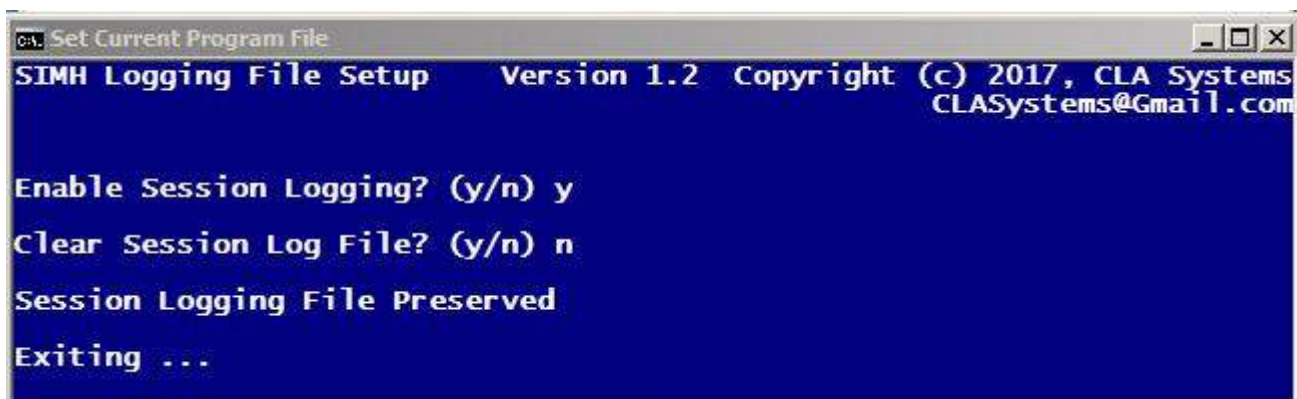


Figure 12. Logging File Setup

Appendix A.

Recovery from accidental deletion of TFS files

As is the case in any file system, it is possible to delete TFS files as the result of unintended acts. Various strategies will be offered to maximize the possibility of proper file recovery.

- 1) Recovering a file accidentally deleted while the file is in the edit buffer.

If the file is still available in the edit buffer, it can be saved to the TFS directory by executing the appropriate file save operation (the SA{ve} command in older releases, the WR{ite} command in newer releases).

For most users of P?S/8, this is the only remedial action needed. However, if file placement is an issue, it may be necessary to place dummy file entries to occupy empty file slots before the intended storage blocks. The CA{talog} command can be used to confirm the existence of empty entries prior to the desired storage blocks. The EN{ter} command can be used as many times as necessary to achieve the desired results. (Note: Precise file placement can be deferred as long as the file is properly saved somewhere within the TFS directory structure.)

- 2) Recovering a file accidentally deleted and the only known copy is stored in a TFS directory slot.

No file recovery is possible if the file slot is overwritten; as long as the file slot is available with the original file contents, recovery is still possible.

If the proper file slot is evident, an EN{ter} command can be used to recover the file contents and original file name. The file must be brought into the edit buffer for verification of the expected contents.

Note: The EN{ter} command does not check the contents of the corresponding file storage blocks. Under perverse circumstances, the system may produce junk output from the LI{st} or related commands, or possibly hang the keyboard monitor program when the contents of the corresponding blocks are loaded based on presumption of file validity that turn out to be wrong. Uninitialized tape blocks do not represent a proper TFS text file. By design, no portion of the system device will be accidentally written in such a case; it will be necessary to restart the system at 07600 or other alternative startup methods.

Appendix A (Continued).

As of this writing, there are no utilities to reveal the name of partially deleted files; future releases of the P?S/8 DIRECT utility will add in the ability to view the file name remnant. (Note: Deleted TFS files do not retain the first two file name characters; any additional characters in the file name can be viewed with a proper utility. In the near-term, advanced users of the P?S/8 DUMP program can carry out file name repairs using the ZAP function.)

If there is sufficient confusion regarding the proper file slot for recovery, the recommended procedure is to fill all available file slots with dummy names using the EN{ter} command as many times as needed. If available, the P?S/8 DIRECT utility can identify the file type of each file by the use of appropriate command-line options. (Note: This operation can take significant time to execute; every file is loaded and checked for internal signatures.)

If desired, all slots in the TFS directory can be filled with dummy names prior to use. This will allow any relative file placement for optimization purposes such as the method detailed in the description of the EN{ter} command.

With every available file slot filled, file recovery will generally become a trivial task; file deletion will readily reveal empty file slots.

Future releases of the FILMAN utility may support aliased entry points for common file operations such as MOVE and RENAME. An UNDELE{TE} alias can be added that could guide the user through file recovery limiting choices to partially matching deleted file names only. (Note: It will be a challenge to implement certain of these features unless the operational name is changed or existing keyboard monitor commands are renamed; however, the internal commands of FILMAN can be expanded to support such operations within the context of the FILMAN utility itself.)

End of Appendix A.

Appendix G - Launching a paper-tape software development session

The paper-tape operating system can be initiated using the shortcut/link provided.

Files can be loaded over the contents of memory; this includes cumulative loading such as is used in the example:

First the binary file containing FOCAL, 1969 is loaded; this is followed by the optional INIT overlay to allow reconfiguration of FOCAL to optimize for either functionality or storage.

The contents of memory can be written to a file as appropriate.

Unlike the actual hardware, files are loaded into SIMH using a "phantom" BIN loader that does not occupy memory.

PTUTIL is a recent addition to later releases of P/S/8 Version 8Z; it will be included in all future releases as a standard system program.

Users familiar with the P/S/8 System Programs Directory can add PTUTIL to existing systems where PTUTIL is not installed (assuming sufficient free blocks are available). PTUTIL can also be run as an impromptu system program as described below; the binary files created by assembling the PTUTIL source files must be available on an accessible logical device unit.

Note: Logical unit 1 is used in this example; appropriate changes can be applied as needed.

Console output

Comments

.	Keyboard monitor prompt.
.CA:1	CA{talog} command for unit 1.
BPTUT1 BPTUT2 README RDMETU	Contents of TFS Directory on unit containing binary files.
.LC:0	Back to system unit to avoid dependencies on data unit.
.GET BPTUT1:1 BPTUT2	Create program image.
.R \$ (Files and other parameters)	System restarts here.
	Particulars same as used when PTUTIL is normally installed.

The R (or RU{n}) command specifying the program name \$ (or %) executes the default core image as setup by the latest GET execution or equivalent. (Note: This will function permanently unless some other system usage or command changes the contents of the % and \$ files. As necessary, the GET operation can be repeated to refresh the default program image.)

PTUTIL has been designed to be (nearly) compatible with the technique described above to allow quick addition to existing systems; this execution mechanism alternative was introduced in P/S/8 Version 8Y and is supported on all newer releases. The majority of conforming system programs can be supported using this tentative mechanism in lieu of inclusion within the System Programs Directory (unless overlay

structures are required).

Appendix A (continued)

Usage restrictions.

When using PTUTIL to create ASCII text paper-tape output, the default leader/trailer length (surrounding the content obtained from all input files) is 100 (0144 octal) frames. This value can be changed using the =xxxx parameter which is normally expressed in the current keyboard monitor radix (which is generally decimal). However, the execution attribute to maintain the current radix is only available to programs located in the System Programs Directory. (The default value is to force octal radix). As such, any usage of the =xxxx parameter as described here must be expressed in octal.

End of Appendix F

Appendix H - Disabling Auto-arrange in newer windows releases

Note: Before attempting any of the changes described here, this entire section should be carefully read and completely understood. It is necessary to have some familiarity with the windows Regedit utility to perform these tasks; as required, seek assistance from a knowledgeable advanced windows user.

As shown in Figure 3 above, it may be desirable to customize the organization of icons within the Command window (and perhaps other windows set to display in any of the available icon views) to enhance readability and lend structure. To accomplish this, clicking on the View command within each window displays separate options for Auto Arrange (which should be disabled) and Align to Grid (which should be enabled); each option is indicated as enabled by a leading check mark where applicable.

Unfortunately, these options were removed from standard release starting with windows 7; in essence, the effect of both of these options is only available together and the separate option check-off display area has been deleted.

A user-written registry patch is available to overcome this limitation which fully restores the classic options as implemented in windows XP and windows Vista. Registry patch files disableautoarrange.reg (and a companion file undoautoarrangedisable.reg to revert windows to the standard release) are available in the Auto-arrange Disable directory within the Links and Support Files directory; the master release file manifest within Appendix Z should be queried to locate these files as required.

There are several steps involved as follows:

- 1) The Regedit registry editor program must be invoked manually to determine which specific override options are present on any particular system. The specifics of system configuration and release may vary; as such, it is necessary to check all potential registry sub-keys that may be present. No windows release has more than three relevant sub-keys; the method used to delete each existing target sub-key is identical in all cases where the sub-key exists. All three potential registry keys must be checked to

ensure all of the potential sub-keys are removed.

Note: For Windows systems with multiple logins, each individual login may include a separate copy of one or more of the affected sub-keys; as such, it may be necessary to repeat the entire process for each login where the modifications described here are desired. The only purpose served by these sub-keys is to impose the release defaults; they can always be safely removed. An alternate registry patch is provided to restore the original registry settings if the changes described here are later deemed undesirable; when this reversion patch is applied, all of the sub-keys are recreated automatically at the next login when attempting to set any of the View options.

Appendix H (Continued).

- 2) After all potential sub-keys described above are deleted, windows must be rebooted. At this point, none of the potential conditions that affect icon placement are in effect. To ensure the options can be applied separately, it is necessary at this point to apply the *disableautoarrange.reg* registry patch file contents; this is generally accomplished by selecting the *Merge* option that appears when performing a right-click on the file name.
- 3) Using Windows Explorer (known in some systems as Windows File Explorer), the main system directory (known as Computer in Windows 7 and This PC in newer editions) should be opened and closed.
- 4) After the system is rebooted, the options should be restored in all icon-based views.

Removing the affected sub-keys.

There are three potential registry sections where these sub-keys may exist:

- 1) HKEY_CURRENT_USER\Software\Microsoft\windows\Shell
- 2) HKEY_CURRENT_USER\Software\Microsoft\windows\ShellNoRoam
- 3) HKEY_CURRENT_USER\Software\Classes\Local Settings\Software\Microsoft\windows\Shell

Using Regedit, navigate to each of these keys and determine if a sub-key called *bags* is present. Highlighting the sub-key allows a right-click menu to appear; select the *Delete* option to delete the sub-key and all data within it. As stated above, all three potential registry areas must be checked for the *bags* sub-key; however, not all systems include all three registry keys. All of the steps need to be followed completely as stated above and in the specified order to achieve success.

Proof of successful operation.

To determine if the sub-key removal and registry patch operations of *disableautoarrange.reg* were successful, open any window and click on the View option. When selecting any icon-based display setting, the two options should be displayed (including the current status as indicated by a leading check mark if enabled). In all other display settings, the two options should be displayed as greyed out. This is totally consistent with the standard implementation available in windows XP and windows Vista.

End of Appendix H.

Notes and References

1. The Poly Question Society (abbreviated wherever possible as P?S, or PQS if only a limited character set is available and pronounced as if spelled pea-queue-ess) is an informal group of questioners concerned with a multitude of subjects. In general, the members of the P?S (whether alive, dead or imaginary) met many years ago at an institution of higher learning known as the Polytechnic Institute of Brooklyn or more informally Brooklyn Poly (now a division of New York University). Nearly all P?S members are computer programmers known for exceptional works implemented for various computer architectures, especially the Digital Equipment Corporation (DEC) PDP-8 family.

The P?S is an outgrowth of a pre-existing organization known as the William Claude Fields Memorial Programming Group (WCFMPG) (also known as Friends of the Boston Massacre). The various works of the WCFMPG are legendary in the realm of PDP-8 programming; all original WCFMPG members were responsible for either starting or creating various software projects for DEC computers, especially the PDP-8. All members of the WCFMPG were DEC employees at some point in their careers; however, much of their work for PDP-8 systems (and other computers made by other companies) predates their DEC employment.

2. SIMH stands for "history simulation" (in the sense of the history of computing) for the purpose of preserving as much as possible the functionality of computers generally no longer manufactured. The SIMH project is maintained by a loose affiliation of programmers, most generally specializing in one or two computer architectures out of the many systems supported.

Levels of support for specific computers varies; at one point the SIMH project was very DEC-centric but now includes support for models from various other manufacturers.

More information on the SIMH project can be obtained from various Internet web sites hosting components of the overall SIMH project including various source code files and other forms of documentation.

3. SIMH simulator support is available for additional PDP-8 features and devices normally disabled by default; however, additional options are documented within the SIMH environment by invoking certain internal help commands. Where relevant, additional documentation is provided above in Appendix A. For those seeking further information, links to certain Internet web sites are provided that may supply information regarding seldom used features of SIMH; however, most users of PEPS will not be concerned unless intending to develop software expressly targeting specific devices.

4. Minor adjustments may be required in certain PEPS files depending on whether the Microsoft Windows edition used is 32-bit or 64-bit.

Where applicable, certain third-party utilities added to the Windows system must be chosen consistently with the edition used when separate installation packages are available.

5. OS/8 is the principal marketing name of an operating system that can run on most PDP-8 models. Note: In some instances, variant systems with slightly different names were created due to the inability to support slightly incompatible hardware; the level of support provided on these systems was quite deficient and generally far less competent than most readers of this document would expect. The design was too inflexible to support these newer systems directly; as such, incomplete and incompatible subsets were released using various marketing names.

This group of name-variant systems constitutes a confusing mess since some of the variations were created merely for the purposes of highly dubious marketing ploys; only an expert can tell the difference between a variant with little or no technical purpose over OS/8 Version 3D (also known as Version 3Q) and another slightly different system that can only be used with newer models such as what are known as Omnibus machines, or perhaps DECmates depending on the specific variations (albeit not necessarily either complete or even completely functional). In general it is prudent to avoid the use of these newer variants under the SIMH simulator; there is a strong likelihood of subtle (and perhaps nasty) incompatibilities appearing seemingly out of nowhere.

Certain specific programs acquired from newer release variants as mentioned above provide acceptable functionality when used in OS/8 V3D. As such, the image copy of OS/8 provided in the PEPS package contains several notable programs that correct bugs and quirks in the originally released versions associated with OS/8 Version V3D.

OS/8 was originally created as a quick-and-dirty hack pulled off over a single weekend intended as a demo program to be shown to DEC management the next Monday morning. While there was every intention to write a second replacement system from scratch with more reasoned internal design, management foolishly insisted all further changes be made to the demo version instead of the intended complete rewrite.

As a direct result of this disastrous blunder, many serious structural problems became institutionalized into the design preventing correction in future work without major rewrites. (Predictably, this never happened.)

The original demo (and some of the follow-on work) was done by Richard Lary, who also created most of the R-L Monitor System (also known later as MS/8) (along with the rest of the members of the WCFMPG). The R-L Monitor System was the first PDP-8 operating system ever used by most of the members of the P?S.

PEPS User Guide Notes and References (Continued).

Part of the reason the deficient demo was created can be traced to the demonstration of the R-L Monitor System to DEC management personnel; all of them rejected the R-L Monitor System essentially because they had no idea what it was or what it actually accomplished. All of them, at one time or another, demonstrated their profound ignorance of how a PDP-8 operating system functions, and in at least one case this directly involved the author of this document. However, this did not stop them from making lunatic demands on how the new system should function in ways analogous to other poor operating systems, especially some superficially applied techniques as used in the Disk Monitor System (which made the design somewhat worse).

The demo was designed to show how a computer user could approach the PDP-8 in a manner vaguely analogous to the way this was accomplished on other DEC computers available at the time (such as the use of TOPS10 on the PDP-10 from the point of view of a time-sharing user). As such, the managers obtained a largely superficial and flawed understanding of the demo (which for them was likely the most they could muster considering their minimal understanding of the other operating systems mentioned above available at the time).

It is noteworthy that all other members of the WCFMPG refused to work on the demo because of its flimsy nature. This is especially important since their collective ideas--hashed out over the course of several months time--was the starting point for many of the features of the intended replacement system, which was now rejected out of hand by the ignorant managers.

Later, some other members of the P?S were hired by DEC to work on OS/8 along with a few unrelated staff members of varying abilities. While this mixed group did eventually develop the demo into something more functional, the inherent internal limitations cannot be overcome without a complete rewrite (as was initially proposed earlier but then rejected by management).

Using OS/8 today, whether on real-world hardware or within the PEPS/SIMH environment, requires mastering certain unexpected skills such as working in a mythical date environment sometime prior to 01-Jan-2002. Unfortunately, due to limitations within the file system, it is not possible to determine the proper year of any file because there is an inherent anomaly regarding which group of eight years in the range of 1970-1977, 1978-1985, 1986-1993 or 1994-2001 is the true file origination date reported by the available utilities in an obfuscated manner.

PEPS User Guide Notes and References (Continued).

In specific instances, certain users can ascertain the proper year grouping for particular files from personal experience. Inevitably, any collection of files will tend to become commingled with files from one of the other three possible date groups; worse still, files created today must have entirely mythical dates. New users must keep track of the real-world date discrepancy on a file by file basis. Because of this serious anomaly, coupled with the advantages offered by standard features of the PEPS package, it is strongly recommended all users embrace host source file editing. Within the host environment, more descriptive filenames are feasible with accurate file dates (as long as the file date can be set after 01-Jan-1980).

Note: Unfortunately, the prevailing host file system limitation prevents proper file date identification regarding files known to have been created between 01-Jan-1970 and 31-Dec-1977 (or possibly earlier on systems such as TOPS-10 which supports dates as early as 1964).

Certain intrepid users are known for manually applying pragmatic date modifications to output obtained from OS/8 directory listing utilities; this additional effort allows the maintenance of textual records beyond the capability of OS/8.

Note: Due to Y2K considerations, this practice has an upper limit of 31-Dec-2069; beyond that date, even the additional manual efforts will break down.

6. P?S/8 is a replacement for the R-L Monitor System. It includes hundreds of improvements applied to new work carried out over the course of decades. Despite all of this effort, the basic P?S/8 system runs with low overhead (and in some ways lower overhead) when compared to the original R-L Monitor System. On certain hardware configurations, P?S/8 runs palpably faster than OS/8 performing comparable tasks.

Certain specific concepts unique to the R-L Monitor System (and not translatable to OS/8) have been faithfully implemented in P?S/8, and in some cases actually improved.

P?S/8 is still currently being developed. While the P?S/8 release included in the current PEPS package is Version 8Z (first released on 23-Feb-1988), various improvements and bug fixes have been applied to the system image included in the PEPS release.

When P?S/8 Version 9A is available, it will replace the present Version 8Z release and be included in future PEPS packages.

PEPS User Guide Notes and References (Continued).

P?S/8 runs on all of the same PDP-8 models that OS/8 (and its variants) runs on; however, a single P?S/8 release covers all models, differing only in terms of specific device handlers and related utilities where applicable. It is noteworthy that a suitably configured P?S/8 system written to RX01 diskette media can be booted to all properly configured systems from the original "straight" PDP-8 through the DECmate II. By way of comparison, three different OS/8 variants are required to support the same models due to insurmountable incompatibilities inherent in OS/8 internal design (and at least four variants were marketed for this purpose).

Note: No OS/8 variant exists supporting bootable RX01 diskettes on the DECmate II.

A minor patch must be applied to the supplied P?S/8 release for the TC01/TC08 DECTape controller due to SIMH incompatibility with the real-world hardware; the P?S/8 keyboard monitor includes direct commands to rewind and unload DECTapes. When applied on the real-world hardware, appropriate use of these commands helps achieve noticeable performance improvements; unfortunately, using these commands is known to destabilize the present SIMH program.

Documentation is available explaining the minor (reversible) patching procedure needed when moving the system image to actual DECTape hardware for use on real-world systems. Various programs exist to move DECTape images between physical DECTapes and the SIMH-compatible files used in the PEPS environment. Appendix D provides links to appropriate utilities for this and related purposes.

7. BIN format is the standard binary loading format defined in the Paper-tape operating system. On occasion, a BIN format paper-tape image file must be loaded into memory in the SIMH simulated environment. Both the OS/8 and P?S/8 operating systems support utilities to process BIN format data passed to the SIMH environment by setting up the PTRIN.TXT file with the intended binary contents. When files are moved from the SIMH environment to the host windows system, the PTPOUT.TXT file (or perhaps the LPTOUT.TXT file) is used for file transfer.

In-progress notes and other junk from here onwards; at this point nearly gone!

<ftp://simh.trailing-edge.com/pub/mirror/www.aracnet.com/~healyzh/pdp8emu.html>

http://simh.trailing-edge.com/pdf/simh_doc.pdf

[End-of-file]